



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Mid-Term Project Report

Summer 2025-2026

Section: B

Group: 08

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: S.M. NAHIAN ID: 23-51270-1	Data Preprocessing, Data Cleaning, Naive bayes with dataset_1: all evaluation metrics
2.	Name: MD. MOSHAROF HOSSAIN KHAN ID: 23-51259-1	Naive bayes with dataset_2, Logistic Regression with dataset_1: all evaluation matrices
3.	Name: TAWFIQUE AHMED SAMRAT ID: 22-46502-1	Linear SVM with dataset_1, Logistic Regression with dataset_2: all evaluation matrices
4.	Name: MD Mazharul Islam Nabil ID: 23-50025-1	Linear SVM with dataset_2 all evaluation matrices, full report writing

Dataset Description

Two datasets were selected for the emotion detection task. Dataset 1 (emotion_dataset_raw.csv) contains 34,792 short social-media style text entries labeled with one of eight emotions: joy, sadness, fear, anger, surprise, neutral, disgust, and shame. Dataset 2 (combined_emotion.csv) contains 422,746 sentences labeled with one of six emotions: joy, sad, fear, anger, love, and surprise. After removing duplicate text and rows left empty by cleaning, Dataset 1 reduced to 30,283 rows and Dataset 2 reduced to 379,611 rows. Both datasets were cleaned using the same preprocessing pipeline (case folding, noise and punctuation removal, tokenization, stopword removal, and stemming) before being used across all three classification algorithms.

Project Implementation Detail

Data Loading and Preprocessing

Task 1: Importing all necessary libraries

Description: All the libraries needed for vectorization, model training, and evaluation across the whole project were imported at the start of the notebook.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

Code description: pandas and numpy handle the data, CountVectorizer and TfidfVectorizer build the Bag of Words and TF-IDF representations, MultinomialNB, LogisticRegression, and LinearSVC are the three required classifiers, and the metric functions are used for evaluation throughout the notebook.

Task 2: Importing datasets and checking their shapes

Description: The two raw datasets were loaded into pandas DataFrames and their shapes were printed to confirm the number of rows and columns.

Code:

```
dataset_1 = pd.read_csv("emotion_dataset_raw.csv")
dataset_2 = pd.read_csv("combined_emotion.csv")
```

```
print(dataset_1.shape)
```

```
print(dataset_2.shape)
```

Output:

```
(34792, 2)
```

```
(422746, 2)
```

Code description: read_csv loads each dataset, and printing the shape confirms Dataset 1 has 34,792 rows and Dataset 2 has 422,746 rows, both with two columns (text and emotion label).

Task 3: Importing libraries for text preprocessing

Description: The libraries needed for cleaning and normalizing the text were imported, along with the required NLTK resources.

Code:

```
import re
```

```
import string
```

```
import nltk
```

```
from nltk.corpus import stopwords
```

```
from nltk.stem import PorterStemmer
```

```
from nltk.tokenize import word_tokenize
```

```
nltk.download('stopwords')
```

```
nltk.download('punkt')
```

Output:

```
[nltk_data] Downloading package stopwords to ... up-to-date!
```

```
[nltk_data] Downloading package punkt to ... up-to-date!
```

```
True
```

Code description: re and string support regular expression and punctuation cleaning, stopwords gives the standard English stopword list, PorterStemmer performs stemming, and word_tokenize performs tokenization, all directly from the Text Normalization lectures.

Dataset 1 Preprocessing

Task 4: Creating a working copy of Dataset 1

Description: A copy of Dataset 1 was made so the original raw data stays untouched while cleaning is performed.

Code:

```
copy_d1 = dataset_1.copy()
```

Code description: Working on a copy keeps the original dataset_1 available for comparison against the cleaned version later.

Task 5: Checking for duplicate text in Dataset 1

Description: The text column of Dataset 1 was checked for duplicate entries before any cleaning was applied.

Code:

```
duplicates = copy_d1['Text'].duplicated()
print(duplicates)
```

```
duplicates = copy_d1['Text'].duplicated().sum()
print(duplicates)
```

Output:

```
0    False
1    False
...     ...
```

```
Name: Text, Length: 34792, dtype: bool
```

```
3682
```

Code description: duplicated() flags repeated text rows, and summing it shows 3,682 duplicate rows exist in the raw text of Dataset 1.

Task 6: Removing duplicate text from Dataset 1 (first pass)

Description: The duplicate rows identified in the previous step were removed from Dataset 1.

Code:

```
copy_d1 = copy_d1.drop_duplicates(subset=['Text'])
```

```
print(copy_d1)
```

Output:

```
      Emotion      Text
...
```

```
[31110 rows x 2 columns]
```

Code description: `drop_duplicates` keeps only the first occurrence of each repeated text, reducing Dataset 1 from 34,792 to 31,110 rows.

Task 7: Checking for missing values in Dataset 1

Description: The Text and Emotion columns were checked for missing values.

Code:

```
has_nan = copy_d1[['Text', 'Emotion']].isna().any()
print(has_nan)
```

Output:

```
Text      False
Emotion   False
dtype: bool
```

Code description: `isna().any()` confirms neither column has missing values, so no rows needed to be dropped for that reason.

Task 8: Checking noise patterns in Dataset 1 before cleaning

Description: The raw text was checked for web links and HTML-style substrings, user mentions, hashtags, HTML entities, and non-ASCII characters, to see how much noise cleaning would need to remove.

Code:

```
mask = copy_d1['Text'].str.contains(r'href|http|https|www|html|amp|nbsp', case=False,
regex=True, na=False).sum()
print(mask)

mask = copy_d1['Text'].str.contains(r'@\w+', case=False, regex=True, na=False).sum()
print(mask)

mask = copy_d1['Text'].str.contains(r'#\w+', case=False, regex=True, na=False).sum()
print(mask)
```

```
mask = copy_d1['Text'].str.contains(r'&\w+;', case=False, regex=True, na=False).sum()
print(mask)

mask = ~copy_d1['Text'].str.isascii()
print(mask.sum())
```

Output:

```
1454
5068
4267
2119
1421
```

Code description: Each str.contains check uses a regular expression, following the Regex lecture, to count how many rows contain that noise pattern: 1,454 rows contain link-like substrings, 5,068 contain @mentions, 4,267 contain #hashtags, 2,119 contain HTML entities, and 1,421 contain non-ASCII characters.

Task 9: Defining the preprocessing function for Dataset 1

Description: A single function was written to apply case folding, noise removal, punctuation removal, digit removal, tokenization, stopword removal, and stemming to each text entry of Dataset 1.

Code:

```
stop_words = set(stopwords.words('english'))

stemmer = PorterStemmer()

HTML_TAGS = re.compile(r'\b(href|http|https|www|html|amp|nbsp)\w*\b')

def preprocess_1(text):
    text = text.lower()
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'&\w+', '', text)
    text = HTML_TAGS.sub(' ', text)
    text = text.encode('ascii', 'ignore').decode('ascii')
    text = text.translate(str.maketrans("", "", string.punctuation))
```



```

text = re.sub(r'\d+', '', text)
text = re.sub(r'\s+', ' ', text).strip()
tokens = word_tokenize(text)
tokens = [t for t in tokens if t not in stop_words]
tokens = [stemmer.stem(t) for t in tokens]
return ' '.join(tokens)

```

Code description: This one function chains every required technique in order: case folding with `lower()`, noise removal with regular expressions and encode/decode to ASCII, punctuation removal with `string.punctuation`, digit removal, tokenization with `word_tokenize`, stopword removal, and stemming with `PorterStemmer`.

Task 10: Downloading the `punkt_tab` tokenizer resource

Description: The `punkt_tab` resource required by `word_tokenize` in newer NLTK versions was downloaded.

Code:

```
nltk.download('punkt_tab')
```

Output:

```
[nltk_data] Downloading package punkt_tab to ... up-to-date!
```

```
True
```

Code description: `word_tokenize` depends on this resource being available locally before it can split text into tokens.

Task 11: Applying preprocessing to Dataset 1

Description: The preprocessing function was applied to every row of the Text column in Dataset 1.

Code:

```

copy_d1["Text"] = copy_d1["Text"].apply(preprocess_1)
print("done")

```

Output:

```
done
```

Code description: `apply` runs `preprocess_1` on every row, replacing the original text with its cleaned, tokenized, stemmed version.

Task 12: Verifying noise patterns are removed from Dataset 1

Description: The same noise checks used before cleaning were run again to confirm the preprocessing function worked as intended.

Code:

```
mask = copy_d1['Text'].str.contains(r'href|http|https|www|html|amp|nbsp', case=False,
regex=True, na=False).sum()

print(mask)

mask = copy_d1['Text'].str.contains(r'@\w+', case=False, regex=True, na=False).sum()

print(mask)

mask = copy_d1['Text'].str.contains(r'#\w+', case=False, regex=True, na=False).sum()

print(mask)

mask = copy_d1['Text'].str.contains(r'&\w+;', case=False, regex=True, na=False).sum()

print(mask)

mask = ~copy_d1['Text'].str.isascii()

print(mask.sum())
```

Output:

```
316
0
0
0
0
```

Code description: Mentions, hashtags, HTML entities, and non-ASCII characters were fully removed. The 316 remaining matches for the first pattern come from ordinary words that happen to contain the substring 'http' or similar inside them after stemming, not actual links, since real URLs were already stripped.

Task 13: Removing empty rows created by preprocessing (Dataset 1)

Description: Rows that became empty strings after cleaning, and any remaining missing values, were identified and removed.

Code:

```
print(copy_d1['Text'].isnull().sum())

print((copy_d1['Text'] == "").sum())
```

```
print(copy_d1[copy_d1['Text'] == ''])
```

```
copy_d1 = copy_d1[copy_d1['Text'] != '']
copy_d1 = copy_d1.reset_index(drop=True)
```

Output:

```
0
```

```
149
```

```
Emotion Text
```

```
0    neutral
```

```
...
```

```
[149 rows x 2 columns]
```

Code description: 149 rows became empty after cleaning, mostly short neutral entries that consisted only of stopwords or punctuation, and these rows were dropped since they carry no usable text for classification.

Task 14: Confirming no empty rows remain in Dataset 1

Description: A final check was run to confirm no empty text rows remain.

Code:

```
print((copy_d1['Text'] == '').sum())
```

Output:

```
0
```

Code description: The result of 0 confirms all empty rows were successfully removed.

Task 15: Comparing original and processed Dataset 1 samples

Description: The first ten rows of the original and the cleaned Dataset 1 were printed side by side to visually confirm the preprocessing worked correctly.

Code:

```
print("Original Dataset: ", dataset_1.head(10))
print("Processed Dataset: ", copy_d1.head(10))
```

Output:

```
Original: 1    joy    Sage Act upgrade on my to do list for tommorow.
```

Processed: 0 joy sage act upgrad list tommorow

(full 10-row comparison printed)

Code description: The comparison shows the pipeline working as intended: capitalization is gone, punctuation is stripped, and words like 'upgrade' and 'tomorrow' are reduced to their stems 'upgrad' and 'tommorow' is kept since it was misspelled in the source text.

Task 16: Checking Dataset 1 shape after preprocessing

Description: The shape of Dataset 1 was checked after cleaning and empty-row removal.

Code:

```
print(copy_d1.shape)
```

Output:

```
(30961, 2)
```

Code description: Dataset 1 now has 30,961 rows, down from 31,110 before this stage, reflecting the 149 empty rows removed.

Task 17: Checking for duplicate text in Dataset 1 after preprocessing

Description: Since preprocessing can make previously different texts identical, the duplicate check was run again after cleaning.

Code:

```
duplicates = copy_d1['Text'].duplicated().sum()
```

```
print(duplicates)
```

Output:

```
678
```

Code description: 678 new duplicate rows appeared after cleaning, since different raw texts collapsed into the same cleaned form once case, punctuation, and stopwords were stripped away.

Task 18: Removing duplicate text from Dataset 1 (second pass)

Description: The new duplicates found after preprocessing were removed.

Code:

```
copy_d1 = copy_d1.drop_duplicates(subset=['Text'])
```

Code description: This keeps only one copy of each cleaned text, avoiding the model seeing the same processed example more than once.

Task 19: Final shape of Dataset 1 after cleaning

Description: The duplicate count and final shape of Dataset 1 were checked to confirm cleaning was complete.

Code:

```
duplicates = copy_d1['Text'].duplicated().sum()
print(duplicates)
```

```
print(copy_d1.shape)
```

Output:

```
0
```

```
(30283, 2)
```

Code description: Dataset 1 is now fully cleaned and deduplicated, ending at 30,283 rows, down from the original 34,792.

Dataset 2 Preprocessing

Task 20: Creating a working copy of Dataset 2

Description: A copy of Dataset 2 was made so the original raw data stays untouched while cleaning is performed.

Code:

```
copy_d2 = dataset_2.copy()
print(copy_d2)
```

Output:

```
      sentence emotion
```

```
0  i just feel really helpless...  fear
```

```
[422746 rows x 2 columns]
```

Code description: This mirrors the same copy-first approach used for Dataset 1.

Task 21: Checking for duplicate text in Dataset 2

Description: The sentence column of Dataset 2 was checked for duplicate entries before cleaning.

Code:

```
duplicates = copy_d2['sentence'].duplicated()
print(duplicates)
```

```
duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)
```

Output:

```
0    False
```

```
...
```

```
Name: sentence, Length: 422746, dtype: bool
```

```
28924
```

Code description: 28,924 duplicate sentences were found in the raw text of Dataset 2.

Task 22: Removing duplicate text from Dataset 2 (first pass)

Description: The duplicate rows identified in the previous step were removed from Dataset 2.

Code:

```
copy_d2 = copy_d2.drop_duplicates(subset=['sentence'])
```

Code description: This reduces Dataset 2 from 422,746 to 393,822 rows, keeping only the first occurrence of each repeated sentence.

Task 23: Checking for missing values in Dataset 2

Description: The sentence and emotion columns were checked for missing values.

Code:

```
has_nan = copy_d2[['sentence', 'emotion']].isna().any()
print(has_nan)
```

Output:

```
sentence    False
```

```
emotion     False
```

```
dtype: bool
```

Code description: Neither column has missing values in Dataset 2.

Task 24: Checking noise patterns in Dataset 2 before cleaning

Description: The same noise checks used for Dataset 1 were run on Dataset 2 to see what cleaning would need to remove.

Code:

```
mask = copy_d2['sentence'].str.contains(r'href|http|https|www|html|amp|nbsp', case=False,
regex=True, na=False).sum()
print(mask)

mask = copy_d2['sentence'].str.contains(r'@\w+', case=False, regex=True, na=False).sum()
print(mask)

mask = copy_d2['sentence'].str.contains(r'#\w+', case=False, regex=True, na=False).sum()
print(mask)

mask = copy_d2['sentence'].str.contains(r'&\w+;', case=False, regex=True, na=False).sum()
print(mask)

mask = ~copy_d2['sentence'].str.isascii()
print(mask.sum())
```

Output:

9244

0

0

0

0

Code description: 9,244 rows contain link-like substrings, while no mentions, hashtags, HTML entities, or non-ASCII characters appear, since Dataset 2 is already plain lowercase sentence text rather than raw social media posts.

Task 25: Defining the preprocessing function for Dataset 2

Description: A preprocessing function was written for Dataset 2, applying the same normalization steps used for Dataset 1 minus the mention and hashtag removal, since this dataset does not contain them.

Code:

```
stop_words = set(stopwords.words('english'))
```

```
stemmer = PorterStemmer()
HTML_TAGS = re.compile(r'\b(href|http|https|www|html|amp|nbsp)\w*\b')
```

```
def preprocess_2(text):
    text = text.lower()
    text = HTML_TAGS.sub(' ', text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = re.sub(r'\d+', "", text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in stop_words]
    tokens = [stemmer.stem(t) for t in tokens]
    return ' '.join(tokens)
```

Code description: This keeps case folding, link-pattern removal, punctuation removal, digit removal, tokenization, stopword removal, and stemming, the same set of techniques applied consistently across both datasets.

Task 26: Applying preprocessing to Dataset 2

Description: The preprocessing function was applied to every row of the sentence column in Dataset 2.

Code:

```
copy_d2['sentence'] = copy_d2['sentence'].apply(preprocess_2)
print("done")
```

Output:

done

Code description: apply runs preprocess_2 across all 393,822 remaining rows of Dataset 2.

Task 27: Verifying noise patterns are removed from Dataset 2

Description: The link-pattern check was run again after cleaning.

Code:

```
mask = copy_d2['sentence'].str.contains(r'href|http|https|www|html|amp|nbsp', case=False,
regex=True, na=False).sum()
```



```
print(mask)
```

Output:

```
2008
```

Code description: As with Dataset 1, the remaining 2008 matches are ordinary stemmed words that happen to contain the substring pattern, not actual links.

Task 28: Removing empty rows created by preprocessing (Dataset 2)

Description: Rows left empty by cleaning were identified and removed.

Code:

```
print(copy_d2['sentence'].isnull().sum())
print((copy_d2['sentence'] == "").sum())
```

```
copy_d2 = copy_d2[copy_d2['sentence'] != ""]
copy_d2 = copy_d2.reset_index(drop=True)
```

Output:

```
0
9
```

Code description: 9 rows became empty after cleaning and were removed, since they contained no usable text.

Task 29: Confirming no empty rows remain in Dataset 2

Description: A final check was run to confirm no empty text rows remain.

Code:

```
print((copy_d2['sentence'] == "").sum())
```

Output:

```
0
```

Code description: The result of 0 confirms all empty rows were removed.

Task 30: Comparing original and processed Dataset 2 samples

Description: The first ten rows of the original and cleaned Dataset 2 were printed side by side.

Code:

```
print("Original Dataset: ", dataset_2.head(10))
print("Processed Dataset: ", copy_d2.head(10))
```

Output:

Original: 0 i just feel really helpless and heavy hearted fear

Processed: 0 feel realli helpless heavi heart fear

(full 10-row comparison printed)

Code description: The comparison shows stopwords such as 'i', 'just', and 'and' removed, and words such as 'really' and 'heavy' reduced to their stems 'realli' and 'heavi'.

Task 31: Checking Dataset 2 shape after preprocessing

Description: The shape of Dataset 2 was checked after cleaning and empty-row removal.

Code:

```
print(copy_d2.shape)
```

Output:

(39381, 2)

Code description: Dataset 2 now has 393,813 rows, reflecting the 9 empty rows removed from the 393,822 rows left after the first deduplication pass.

Task 32: Checking for duplicate text in Dataset 2 after preprocessing

Description: The duplicate check was run again after cleaning, since preprocessing can make different raw sentences identical.

Code:

```
duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)
```

Output:

14202

Code description: 14,202 new duplicate rows appeared after cleaning, a much larger number than Dataset 1 saw, since Dataset 2 is far bigger and many sentences differ only in stopwords, punctuation, or minor wording that cleaning removes.

Task 33: Removing duplicate text from Dataset 2 (second pass)

Description: The new duplicates found after preprocessing were removed.

Code:

```
copy_d2 = copy_d2.drop_duplicates(subset=['sentence'])
```

Code description: This keeps only one copy of each cleaned sentence.

Task 34: Final shape of Dataset 2 after cleaning

Description: The duplicate count and final shape of Dataset 2 were checked to confirm cleaning was complete.

Code:

```
duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)
```

```
print(copy_d2.shape)
```

Output:

```
0
```

```
(379811, 2)
```

Code description: Dataset 2 is now fully cleaned and deduplicated, ending at 379,811 rows, down from the original 422,746. Both datasets are now ready to be split and vectorized for each of the three classification algorithms.

Naive Bayes - Dataset 1

Task 35: Splitting Dataset 1 into training and testing sets (Naive Bayes)

Description: The cleaned Dataset 1 was split into 80 percent training data and 20 percent testing data for the Multinomial Naive Bayes classifier.

Code:

```
XN_1 = copy_d1['Text']
```

```
YN_1 = copy_d1['Emotion']
```

```
from sklearn.model_selection import train_test_split
```

```
XN1_train, XN1_test, YN1_train, YN1_test = train_test_split(XN_1, YN_1, test_size=0.2,
random_state=42, stratify=YN_1)
```

Code description: train_test_split creates the training and testing sets, and stratify keeps each emotion class in the same proportion across both parts.

Task 36: Bag of Words vectorization - Dataset 1 (Naive Bayes)

Description: The training and testing text of Dataset 1 was converted into Bag of Words feature vectors.

Code:

```
Bag_o_w = CountVectorizer()
XN_train_bow = Bag_o_w.fit_transform(XN1_train)
XN_test_bow = Bag_o_w.transform(XN1_test)
```

Code description: CountVectorizer is fit only on the training text, and the same vocabulary transforms the test text.

Task 37: Training Multinomial Naive Bayes with Bag of Words and evaluating - Dataset 1

Description: A MultinomialNB model was trained on the Bag of Words vectors, used to predict the test labels, and scored with accuracy, precision, recall, and F1.

Code:

```
MNB_BW = MultinomialNB()
MNB_BW.fit(XN_train_bow, YN1_train)

predictions_BW = MNB_BW.predict(XN_test_bow)
print("Accuracy score: ", accuracy_score(YN1_test, predictions_BW))
print("Precision: ", precision_score(YN1_test, predictions_BW, average='weighted'))
print("Recall: ", recall_score(YN1_test, predictions_BW, average='weighted'))
print("F1 Score: ", f1_score(YN1_test, predictions_BW, average='weighted'))
```

Output:

```
Accuracy score: 0.5563810467228001
Precision: 0.5848819678792966
Recall: 0.5563810467228001
F1 Score: 0.5212632964009482
```

Code description: MultinomialNB learns word-frequency probabilities per class, fit trains it, and predict labels the test documents. All four metrics use weighted averaging, which accounts for the class imbalance in Dataset 1.

Task 38: Confusion matrix for the Bag of Words model - Dataset 1 (Naive Bayes)

Description: A confusion matrix was generated for the Bag of Words Naive Bayes model.

Code:

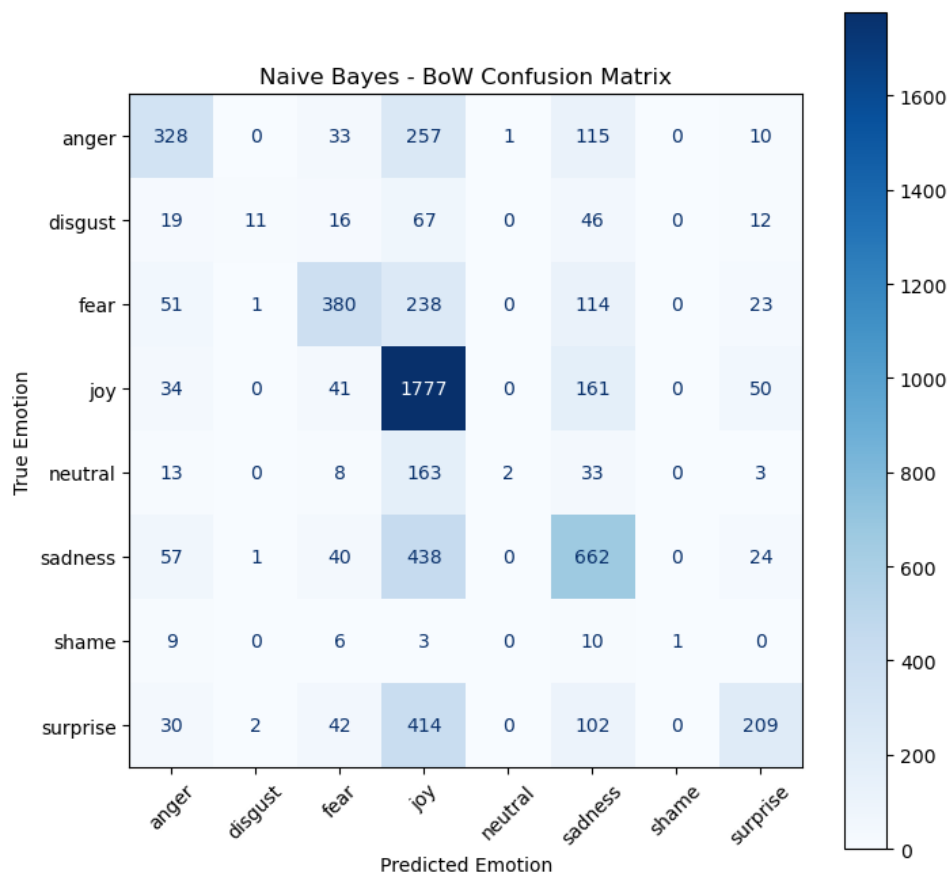
```
emo_labels = sorted(YN1_test.unique())

NBcm_bow = confusion_matrix(YN1_test, predictions_BW, labels=emo_labels)

NBcm_display_bow = ConfusionMatrixDisplay(confusion_matrix=NBcm_bow,
display_labels=emo_labels)

fig, ax = plt.subplots(figsize=(8, 8))
NBcm_display_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Naive Bayes - BoW Confusion Matrix")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```

Output:



Code description: The confusion matrix shows, for each true emotion, how many test documents were predicted into each class, with correct predictions on the diagonal.

Task 39: TF-IDF vectorization - Dataset 1 (Naive Bayes)

Description: The training and testing text of Dataset 1 was converted into TF-IDF feature vectors.

Code:

```
TF_IDF_c = TfidfVectorizer()
XN_train_TFIDF = TF_IDF_c.fit_transform(XN1_train)
XN_test_TFIDF = TF_IDF_c.transform(XN1_test)
```

Code description: TfidfVectorizer is fit only on the training text, mirroring the Bag of Words step.

Task 40: Training Multinomial Naive Bayes with TF-IDF and evaluating - Dataset 1

Description: A MultinomialNB model was trained on the TF-IDF vectors, used to predict the test labels, and scored with the same four metrics.

Code:

```

MNB_TFIDF = MultinomialNB()
MNB_TFIDF.fit(XN_train_TFIDF, YN1_train)

predictions_TFIDF = MNB_TFIDF.predict(XN_test_TFIDF)
print("Accuracy score: ", accuracy_score(YN1_test, predictions_TFIDF))
print("Precision: ", precision_score(YN1_test, predictions_TFIDF, average='weighted'))
print("Recall: ", recall_score(YN1_test, predictions_TFIDF, average='weighted'))
print("F1 Score: ", f1_score(YN1_test, predictions_TFIDF, average='weighted'))

```

Output:

```

Accuracy score: 0.47036486709592207
Precision: 0.5820018828221848
Recall: 0.47036486709592207
F1 Score: 0.4019746575054667

```

UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples.

Code description: The four print statements compute and display accuracy, precision, recall, and F1 by comparing predictions_TFIDF against the true test labels YN1_test, using weighted averaging across classes. scikit-learn prints a warning automatically whenever precision cannot be computed for a class that received no predicted samples. It simply means it ignored the rare classes such as neutral, disgust, and shame.

Task 41: Inspecting actual vs predicted class counts - Dataset 1 TF-IDF (Naive Bayes)

Description: The true and predicted class counts were compared directly to understand the warning from the previous step.

Code:

```

from collections import Counter

print("Actual classes:")
print(Counter(YN1_test))

print("Predicted classes:")

```

```
print(Counter(predictions_TFIDF))
```

Output:

Actual classes: Counter({'joy': 2063, 'sadness': 1222, 'fear': 807, 'surprise': 799, 'anger': 744, 'neutral': 222, 'disgust': 171, 'shame': 29})

Predicted classes: Counter({'joy': 4837, 'sadness': 684, 'fear': 220, 'surprise': 189, 'anger': 127})

Code description: Counter, imported from the collections module, tallies how many times each emotion label appears in the true test labels and in the predicted labels, so the two frequency counts can be printed and compared directly.

Task 42: Confusion matrix for the TF-IDF model - Dataset 1 (Naive Bayes)

Description: A confusion matrix was generated for the TF-IDF Naive Bayes model.

Code:

```
emo_labels = sorted(YN1_test.unique())
```

```
NBcm_tfidf = confusion_matrix(YN1_test, predictions_TFIDF, labels=emo_labels)
```

```
NBcm_display_tfidf = ConfusionMatrixDisplay(confusion_matrix=NBcm_tfidf,
display_labels=emo_labels)
```

```
fig, ax = plt.subplots(figsize=(8, 8))
```

```
NBcm_display_tfidf.plot(ax=ax, cmap="Blues", xticks_rotation=45)
```

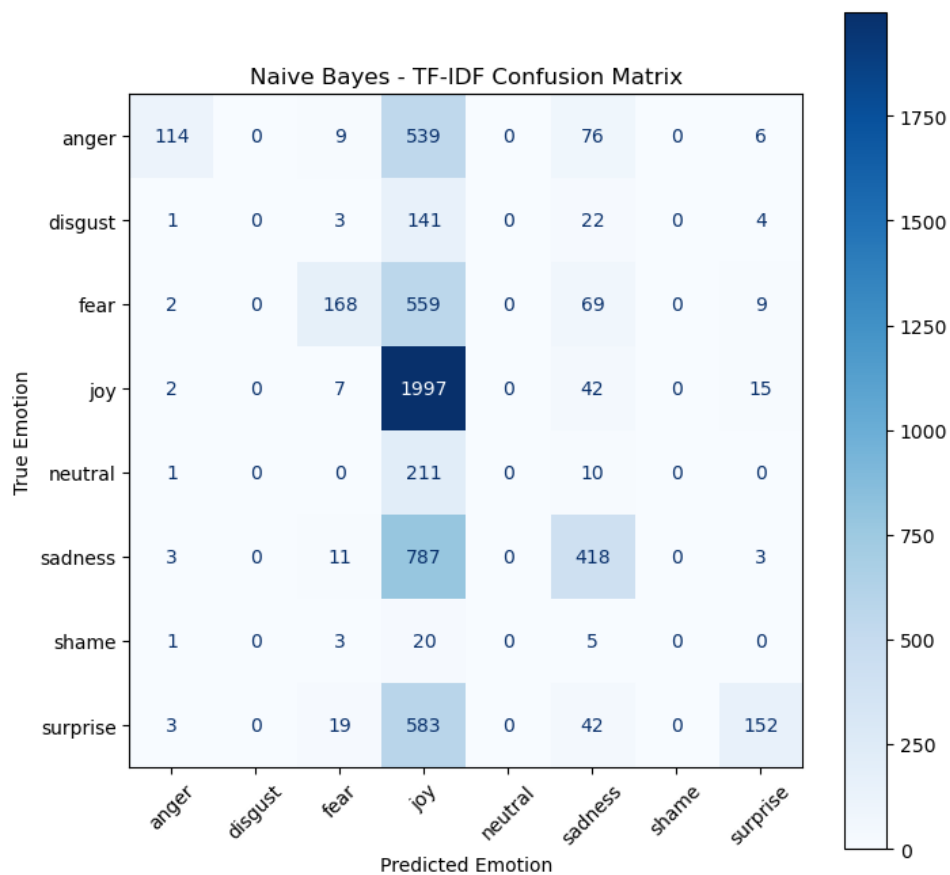
```
plt.title("Naive Bayes - TF-IDF Confusion Matrix")
```

```
plt.xlabel("Predicted Emotion")
```

```
plt.ylabel("True Emotion")
```

```
plt.show()
```

Output:



Code description: `confusion_matrix` builds the matrix of true versus predicted labels using the sorted emotion labels, and `ConfusionMatrixDisplay` renders it as a heatmap with the predicted emotion on the x-axis and the true emotion on the y-axis.

Naive Bayes - Dataset 2

Task 43: Splitting Dataset 2 into training and testing sets (Naive Bayes)

Description: The cleaned Dataset 2 was split into 80 percent training data and 20 percent testing data for the Multinomial Naive Bayes classifier.

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

xt = copy_d2["sentence"]
yt = copy_d2["emotion"]

xt_train, xt_test, yt_train, yt_test = train_test_split(xt, yt, test_size=0.2, random_state=42,
stratify=yt)
```

Code description: The same stratified 80:20 split settings used for Dataset 1 were applied to Dataset 2.

Task 44: Bag of Words vectorization - Dataset 2 (Naive Bayes)

Description: The training and testing text of Dataset 2 was converted into Bag of Words feature vectors.

Code:

```
bow = CountVectorizer()
xt_train_bow = bow.fit_transform(xt_train)
xt_test_bow = bow.transform(xt_test)
```

Code description: The vectorizer is fit only on the training text of Dataset 2.

Task 45: Training Multinomial Naive Bayes with Bag of Words - Dataset 2

Description: A MultinomialNB model was trained on the Bag of Words vectors of Dataset 2 and used to predict the test labels.

Code:

```
MNB_BW = MultinomialNB()
MNB_BW.fit(xt_train_bow, yt_train)

yt_pred_bow = MNB_BW.predict(xt_test_bow)
```

Code description: This mirrors the Dataset 1 Naive Bayes training step, trained and evaluated independently on Dataset 2.

Task 46: TF-IDF vectorization - Dataset 2 (Naive Bayes)

Description: The training and testing text of Dataset 2 was converted into TF-IDF feature vectors.

Code:

```
tfidf = TfidfVectorizer()
xt_train_tfidf = tfidf.fit_transform(xt_train)
xt_test_tfidf = tfidf.transform(xt_test)
```

Code description: The TF-IDF vectorizer is fit only on the training text of Dataset 2.

Task 47: Training Multinomial Naive Bayes with TF-IDF - Dataset 2

Description: A MultinomialNB model was trained on the TF-IDF vectors of Dataset 2 and used to predict the test labels.

Code:

```
MNB_TFIDF = MultinomialNB()
MNB_TFIDF.fit(xt_train_tfidf, yt_train)

yt_pred_tfidf = MNB_TFIDF.predict(xt_test_tfidf)
```

Code description: This mirrors the Bag of Words training step so the two representations can be compared on identical data.

Task 48: Evaluation of the Bag of Words and TF-IDF models - Dataset 2 (Naive Bayes)

Description: Both Naive Bayes models for Dataset 2 were scored on the test set using accuracy, precision, recall, and F1.

Code:

```
accuracy_bow = accuracy_score(yt_test, yt_pred_bow)
accuracy_tfidf = accuracy_score(yt_test, yt_pred_tfidf)
precision_bow = precision_score(yt_test, yt_pred_bow, average='weighted')
precision_tfidf = precision_score(yt_test, yt_pred_tfidf, average='weighted')
recall_bow = recall_score(yt_test, yt_pred_bow, average='weighted')
recall_tfidf = recall_score(yt_test, yt_pred_tfidf, average='weighted')
f1_bow = f1_score(yt_test, yt_pred_bow, average='weighted')
f1_tfidf = f1_score(yt_test, yt_pred_tfidf, average='weighted')

print("Using Bag of Words:")
print(f'Accuracy:{accuracy_bow:.2%}')
print(f'Precision:{precision_bow:.2%}')
print(f'Recall:{recall_bow:.2%}')
print(f'F1-Score:{f1_bow:.2%}')
print("\nUsing TF-IDF:")
print(f'Accuracy:{accuracy_tfidf:.2%}')
```

```
print(f'Precision:{precision_tfidf:.2%}')
print(f'Recall:{recall_tfidf:.2%}')
print(f'F1-Score:{f1_tfidf:.2%}')
```

Output:

Using Bag of Words:

Accuracy:85.89%

Precision:86.13%

Recall:85.89%

F1-Score:85.25%

Using TF-IDF:

Accuracy:75.84%

Precision:80.76%

Recall:75.84%

F1-Score:72.59%

Code description: accuracy_score, precision_score, recall_score, and f1_score are each called once for the Bag of Words predictions and once for the TF-IDF predictions, with average='weighted' so the class sizes are accounted for, then printed with percentage formatting.

Task 49: Detailed classification report - Dataset 2 (Naive Bayes)

Description: A per-class classification report was printed for both representations.

Code:

```
print("Using Bag of Words:")
print(classification_report(yt_test, yt_pred_bow))
print("\nUsing TF-IDF:")
print(classification_report(yt_test, yt_pred_tfidf))
```

Output:

Using Bag of Words: weighted avg precision 0.86 recall 0.86 f1-score 0.85 support 75963

surprise: precision 0.93 recall 0.36 f1-score 0.52

Using TF-IDF: weighted avg precision 0.81 recall 0.76 f1-score 0.73 support 75963

surprise: precision 1.00 recall 0.06 f1-score 0.10

Code description: `classification_report` generates a per-class breakdown of precision, recall, and F1-score together with the support (the number of true samples) for each emotion, along with the overall accuracy and the macro and weighted averages.

Task 50: Confusion matrix for the Bag of Words and TF-IDF models - Dataset 2 (Naive Bayes)

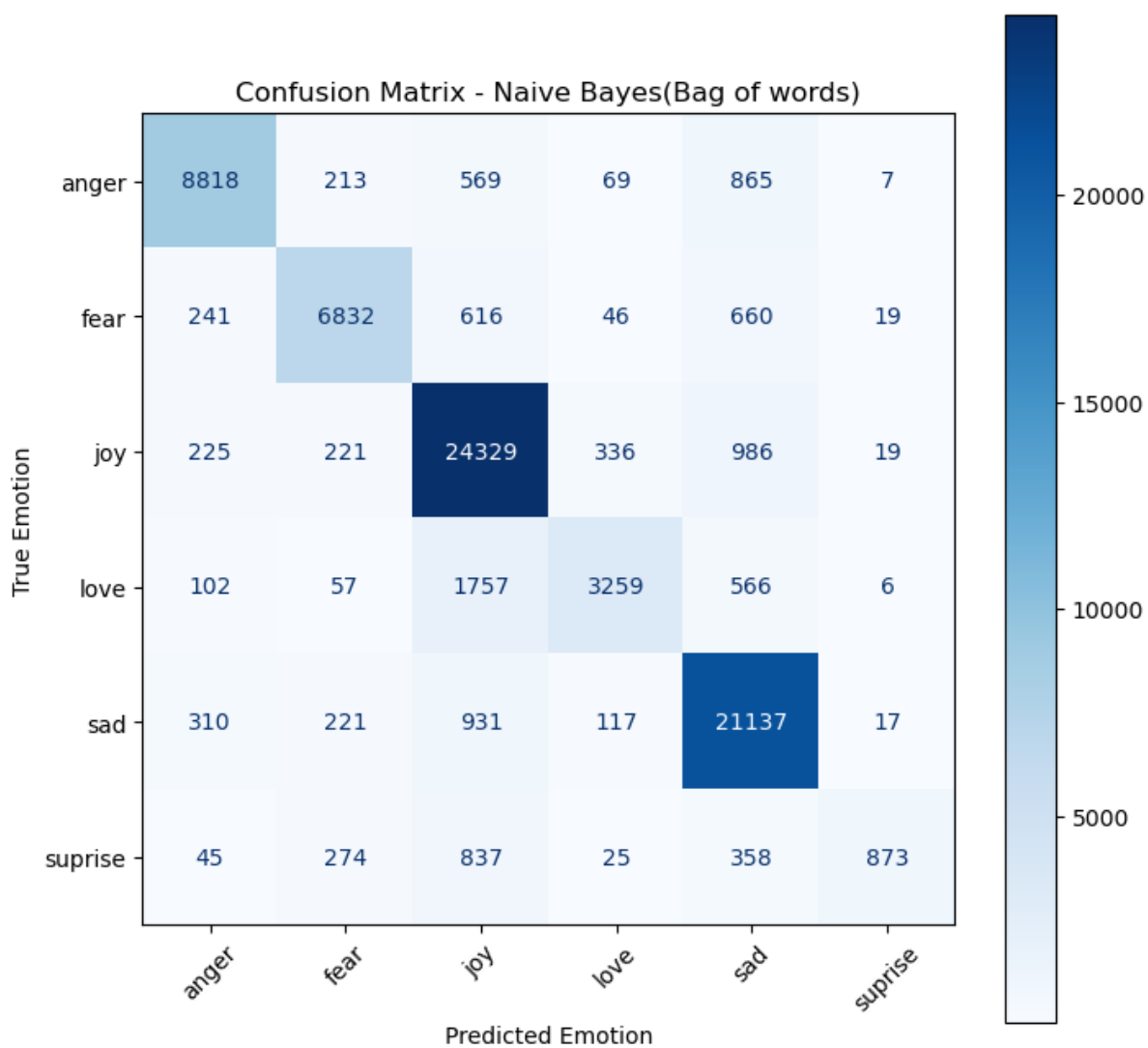
Description: A confusion matrix was generated for each representation on Dataset 2.

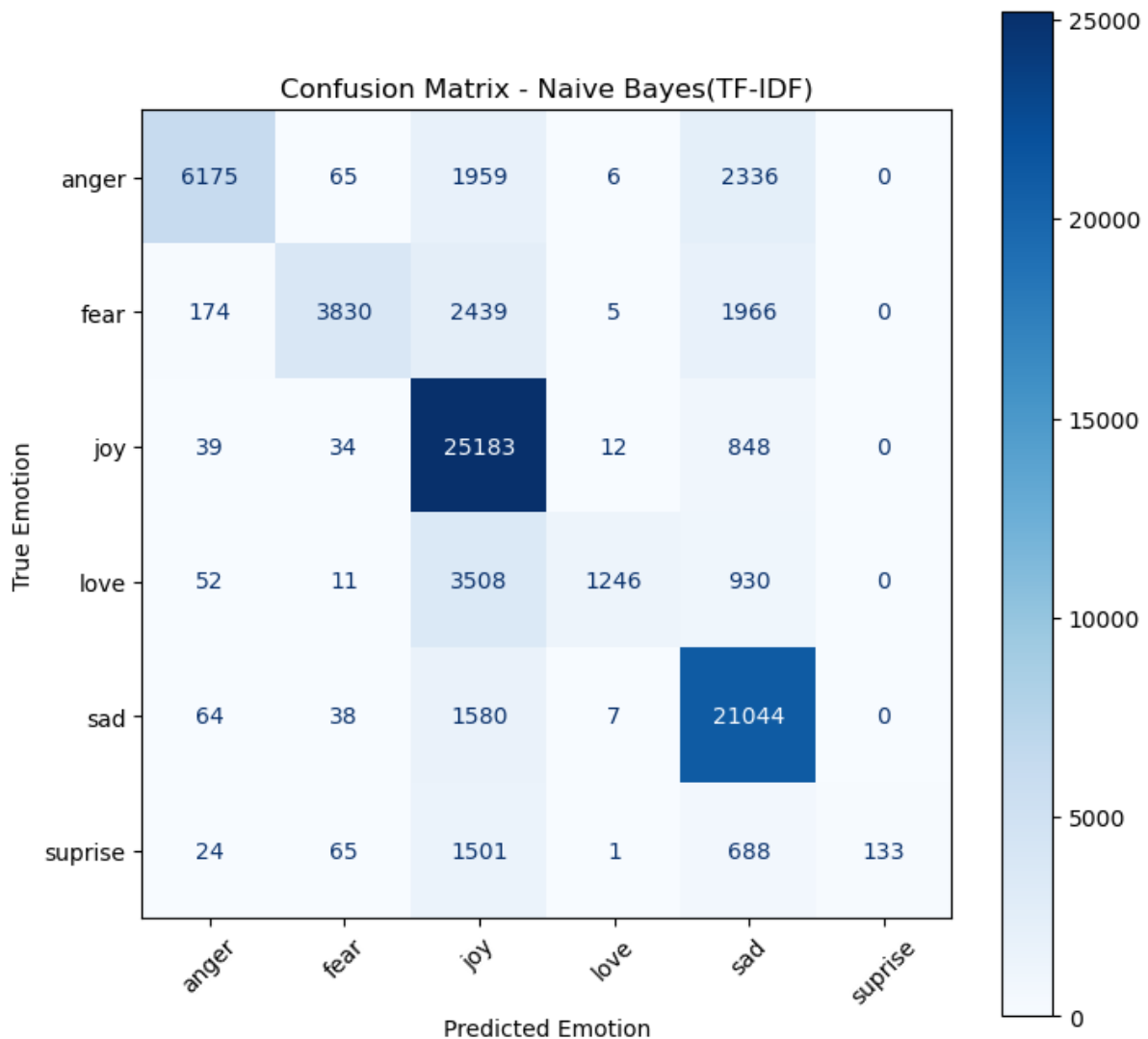
Code:

```
emo_labels = sorted(copy_d2["emotion"].unique())
cm_bow = confusion_matrix(yt_test, yt_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Naive Bayes(Bag of words)")
plt.show()
```

```
cm_tfidf = confusion_matrix(yt_test, yt_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Naive Bayes(TF-IDF)")
plt.show()
```

Output:





Code description: A confusion matrix and heatmap display are built separately for the Bag of Words and TF-IDF predictions, using the same sorted emo_labels list for both so the axis ordering is consistent between the two plots.

Logistic Regression - Dataset 1

Task 51: Splitting Dataset 1 into training and testing sets (Logistic Regression)

Description: The cleaned Dataset 1 was split into 80 percent training data and 20 percent testing data for Logistic Regression.

Code:

```
x = copy_d1["Text"]
y = copy_d1["Emotion"]
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42,
stratify=y)
```

Code description: The same stratified 80:20 split settings used throughout the project were applied here for Logistic Regression.

Task 52: Bag of Words vectorization - Dataset 1 (Logistic Regression)

Description: The training and testing text of Dataset 1 was converted into Bag of Words feature vectors.

Code:

```
bow = CountVectorizer()
x_train_bow = bow.fit_transform(x_train)
x_test_bow = bow.transform(x_test)
```

Code description: CountVectorizer is fit on the training text only, and the test text is transformed using that vocabulary.

Task 53: TF-IDF vectorization - Dataset 1 (Logistic Regression)

Description: The training and testing text of Dataset 1 was converted into TF-IDF feature vectors.

Code:

```
tfidf = TfidfVectorizer()
x_train_tfidf = tfidf.fit_transform(x_train)
x_test_tfidf = tfidf.transform(x_test)
```

Code description: TfidfVectorizer is fit only on the training text of Dataset 1, mirroring the Bag of Words step.

Task 54: Training Logistic Regression with Bag of Words - Dataset 1

Description: A LogisticRegression model was trained on the Bag of Words vectors of Dataset 1.

Code:

```
lr_bow = LogisticRegression(max_iter=1000, random_state=42)
lr_bow.fit(x_train_bow, y_train)
```

Output:

```
LogisticRegression(max_iter=1000, random_state=42)
```


Code description: max_iter was raised to 1000 so the solver has enough iterations to converge on this vocabulary size.

Task 55: Training Logistic Regression with TF-IDF - Dataset 1

Description: A separate LogisticRegression model was trained on the TF-IDF vectors of Dataset 1.

Code:

```
lr_tfidf = LogisticRegression(max_iter=1000, random_state=42)
```

```
lr_tfidf.fit(x_train_tfidf, y_train)
```

Output:

```
LogisticRegression(max_iter=1000, random_state=42)
```

Code description: A separate model object is used here, so the Bag of Words model trained in the previous task is not overwritten.

Task 56: Predicting with both Logistic Regression models - Dataset 1

Description: Each trained model was used to predict the emotion label of the test documents under its own representation.

Code:

```
y_pred_bow = lr_bow.predict(x_test_bow)
```

```
y_pred_tfidf = lr_tfidf.predict(x_test_tfidf)
```

Code description: Because two separate model objects were used, both predictions here are valid and independently comparable.

Task 57: Evaluation of the Bag of Words and TF-IDF models - Dataset 1 (Logistic Regression)

Description: Both Logistic Regression models for Dataset 1 were scored on the test set using accuracy, precision, recall, and F1.

Code:

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)
```

```
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)
```

```
precision_bow = precision_score(y_test, y_pred_bow, average = "weighted")
```

```
precision_tfidf = precision_score(y_test, y_pred_tfidf, average = "weighted")
```

```
recall_bow = recall_score(y_test, y_pred_bow, average = "weighted")
```

```

recall_tfidf = recall_score(y_test, y_pred_tfidf, average = "weighted")
f1_bow = f1_score(y_test, y_pred_bow, average = "weighted")
f1_tfidf = f1_score(y_test, y_pred_tfidf, average = "weighted")

print("Using Bag of Words:")
print(f'Accuracy:{accuracy_bow:.2%}')
print(f'Precision:{precision_bow:.2%}')
print(f'Recall:{recall_bow:.2%}')
print(f'F1-Score:{f1_bow:.2%}')
print("\nUsing TF-IDF:")
print(f'Accuracy:{accuracy_tfidf:.2%}')
print(f'Precision:{precision_tfidf:.2%}')
print(f'Recall:{recall_tfidf:.2%}')
print(f'F1-Score:{f1_tfidf:.2%}')

```

Output:

Using Bag of Words:

Accuracy:58.40%

Precision:57.88%

Recall:58.40%

F1-Score:57.33%

Using TF-IDF:

Accuracy:58.28%

Precision:59.29%

Recall:58.28%

F1-Score:55.94%

Code description: Each metric is computed once for the Bag of Words predictions and once for the TF-IDF predictions, then printed with percentage formatting using an f-string.

Task 58: Confusion matrix for the Bag of Words and TF-IDF models - Dataset 1 (Logistic Regression)

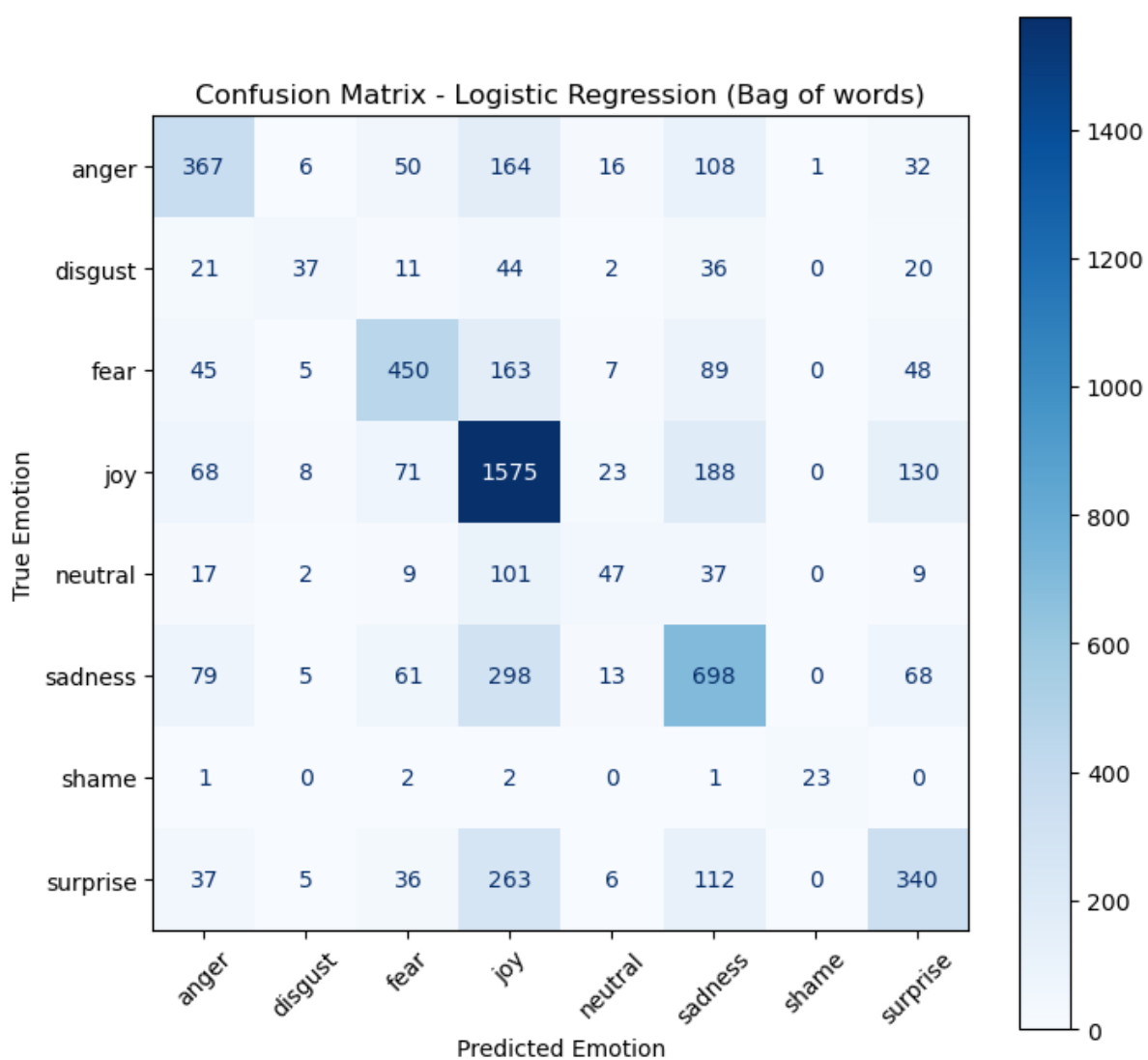
Description: A confusion matrix was generated for each representation on Dataset 1.

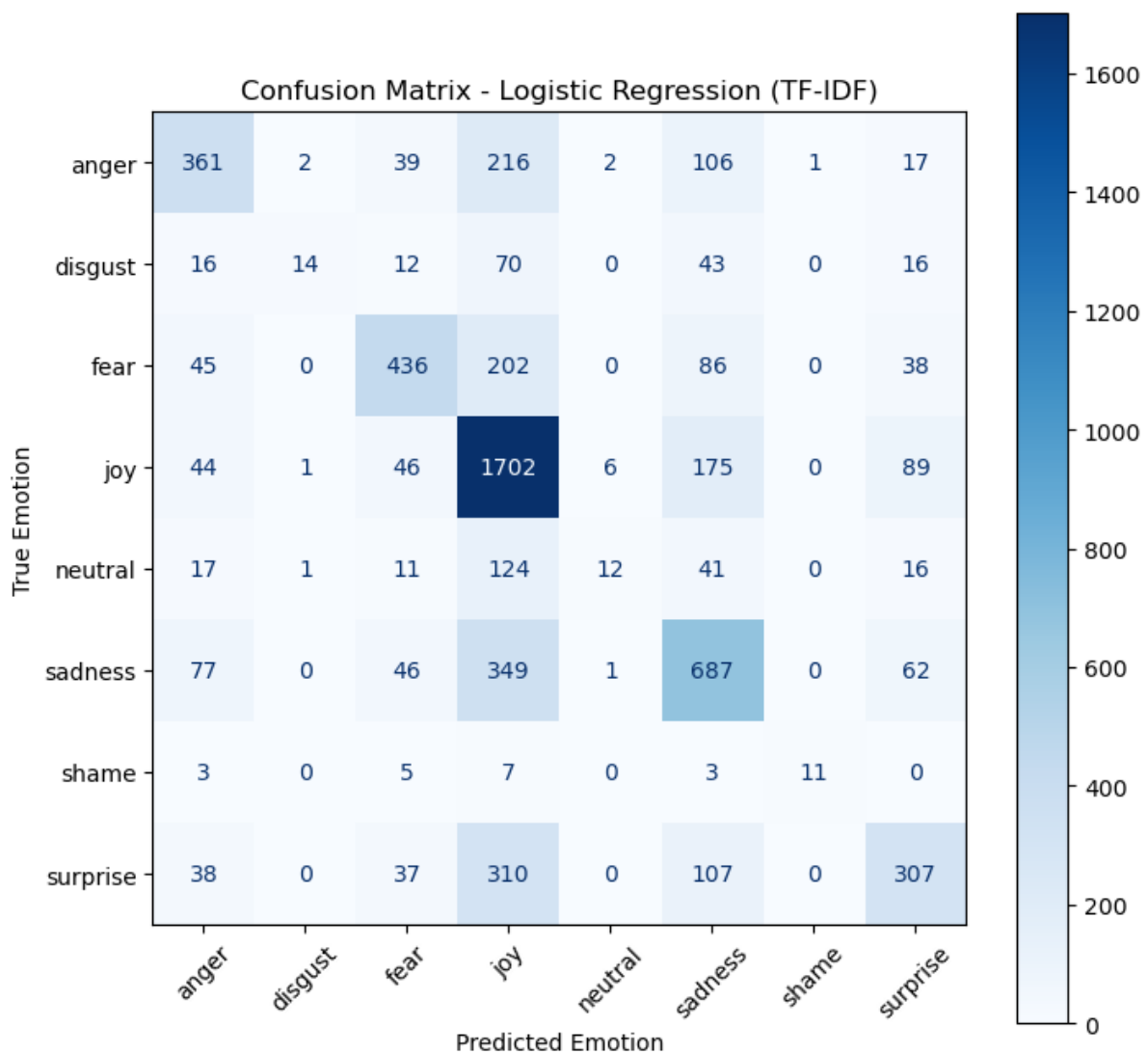
Code:

```
emo_labels = sorted(copy_d1["Emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Logistic Regression (Bag of words)")
plt.show()
```

```
cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Logistic Regression (TF-IDF)")
plt.show()
```

Output:





Code description: Two confusion matrices are built and plotted, one for each representation, using the same sorted list of emotion labels so both figures share the same axis ordering.

Linear Support Vector Classifier - Dataset 1

Task 59: Splitting Dataset 1 into training and testing sets (Linear SVC)

Description: The cleaned Dataset 1 was split into 80 percent training data and 20 percent testing data for the Linear Support Vector Classifier.

Code:

```
from sklearn.model_selection import train_test_split
```

```
x = copy_d1["Text"]
```

```
y = copy_d1["Emotion"]
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state=42,
stratify=y)
```

Code description: The same stratified 80:20 split settings used throughout the project were applied here.

Task 60: Bag of Words vectorization - Dataset 1 (Linear SVC)

Description: The training and testing text of Dataset 1 was converted into Bag of Words feature vectors.

Code:

```
bow = CountVectorizer()
x_train_bow = bow.fit_transform(x_train)
x_test_bow = bow.transform(x_test)
```

Code description: The vectorizer is fit only on the training text of Dataset 1.

Task 61: TF-IDF vectorization - Dataset 1 (Linear SVC)

Description: The training and testing text of Dataset 1 was converted into TF-IDF feature vectors.

Code:

```
tfidf = TfidfVectorizer()
x_train_tfidf = tfidf.fit_transform(x_train)
x_test_tfidf = tfidf.transform(x_test)
```

Code description: The TF-IDF vectorizer is fit only on the training text of Dataset 1.

Task 62: Training the Linear SVC model - Dataset 1

Description: A LinearSVC model was created and fit, first on the Bag of Words vectors and then on the TF-IDF vectors.

Code:

```
svc = LinearSVC(random_state=42)

svc.fit(x_train_bow, y_train)
svc.fit(x_train_tfidf, y_train)
```

Output:

```
LinearSVC(random_state=42)
```

Code description: LinearSVC(random_state=42) creates the classifier with a fixed random state for reproducible results, and fit trains it on the given feature vectors together with their corresponding labels.

Task 63: Predicting with the Linear SVC model - Dataset 1

Description: The fitted model was used to predict the emotion label of the test documents under both representations.

Code:

```
y_pred_bow = svc.predict(x_test_bow)
```

```
y_pred_tfidf = svc.predict(x_test_tfidf)
```

Code description: predict takes the trained model and assigns an emotion label to each document in the test set, once using the Bag of Words test vectors and once using the TF-IDF test vectors.

Task 64: Evaluation of the Bag of Words and TF-IDF results - Dataset 1 (Linear SVC)

Description: Both results for Dataset 1 were scored on the test set using accuracy, precision, recall, and F1.

Code:

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)
```

```
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)
```

```
precision_bow = precision_score(y_test, y_pred_bow, average = "weighted")
```

```
precision_tfidf = precision_score(y_test, y_pred_tfidf, average = "weighted")
```

```
recall_bow = recall_score(y_test, y_pred_bow, average = "weighted")
```

```
recall_tfidf = recall_score(y_test, y_pred_tfidf, average = "weighted")
```

```
f1_bow = f1_score(y_test, y_pred_bow, average = "weighted")
```

```
f1_tfidf = f1_score(y_test, y_pred_tfidf, average = "weighted")
```

```
print("Using Bag of Words:")
```

```
print(f'Accuracy:{accuracy_bow:.2%}')
```

```
print(f'Precision:{precision_bow:.2%}')
```

```

print(f'Recall:{recall_bow:.2%}')
print(f'F1-Score:{f1_bow:.2%}')
print("\nUsing TF-IDF:")
print(f'Accuracy:{accuracy_tfidf:.2%}')
print(f'Precision:{precision_tfidf:.2%}')
print(f'Recall:{recall_tfidf:.2%}')
print(f'F1-Score:{f1_tfidf:.2%}')

```

Output:

Using Bag of Words:

Accuracy:56.13%

Precision:55.74%

Recall:56.13%

F1-Score:55.73%

Using TF-IDF:

Accuracy:58.11%

Precision:57.34%

Recall:58.11%

F1-Score:57.11%

Code description: Each metric is computed once for the Bag of Words predictions and once for the TF-IDF predictions, then printed with percentage formatting.

Task 65: Confusion matrix for the Bag of Words and TF-IDF results - Dataset 1 (Linear SVC)

Description: A confusion matrix was generated for each representation on Dataset 1.

Code:

```

emo_labels = sorted(copy_d1["Emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Linear SVC (Bag of words)")

```



```
plt.show()
```

```
cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
```

```
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,  
display_labels=emo_labels)
```

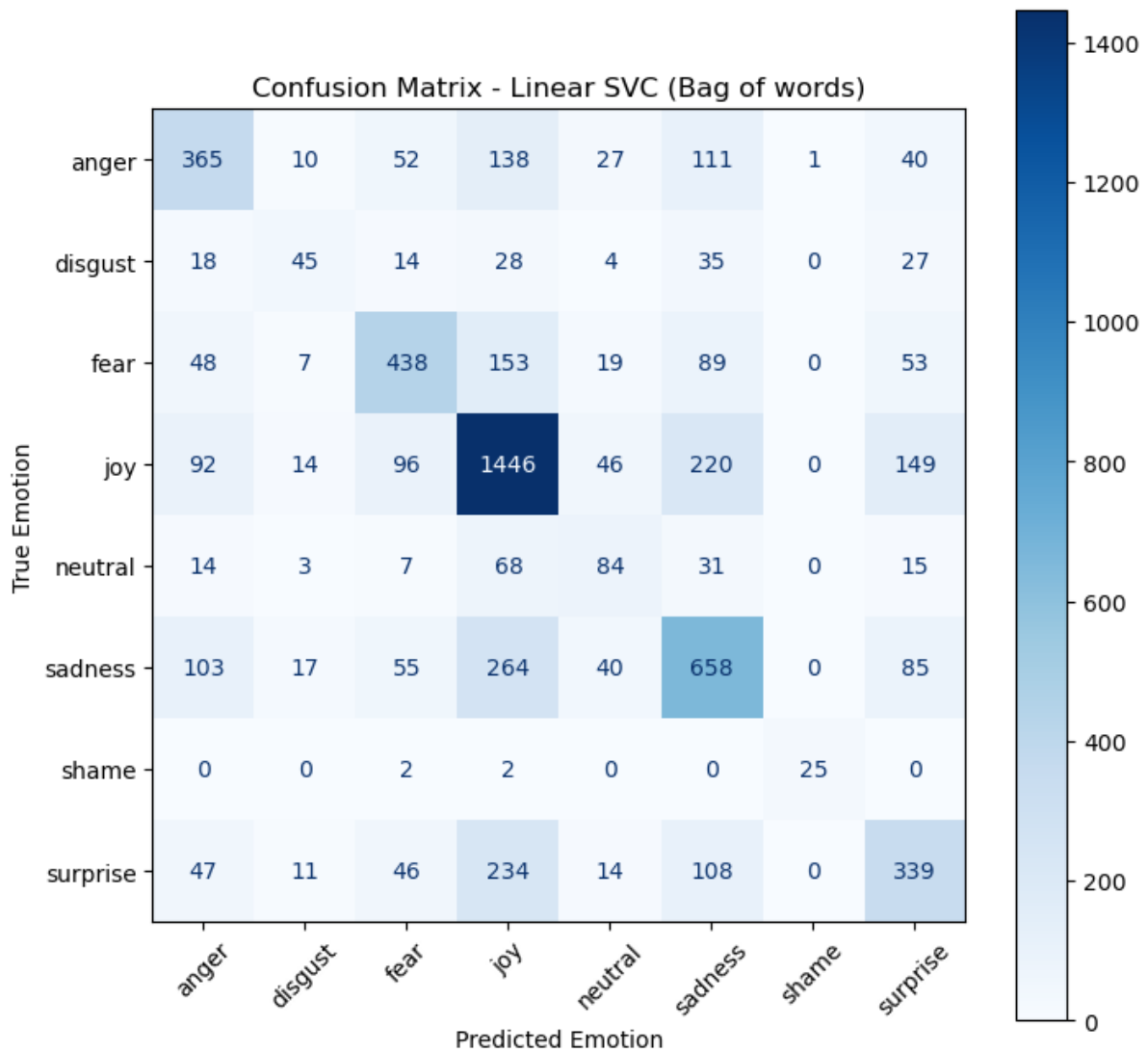
```
fig, ax = plt.subplots(figsize=(8, 8))
```

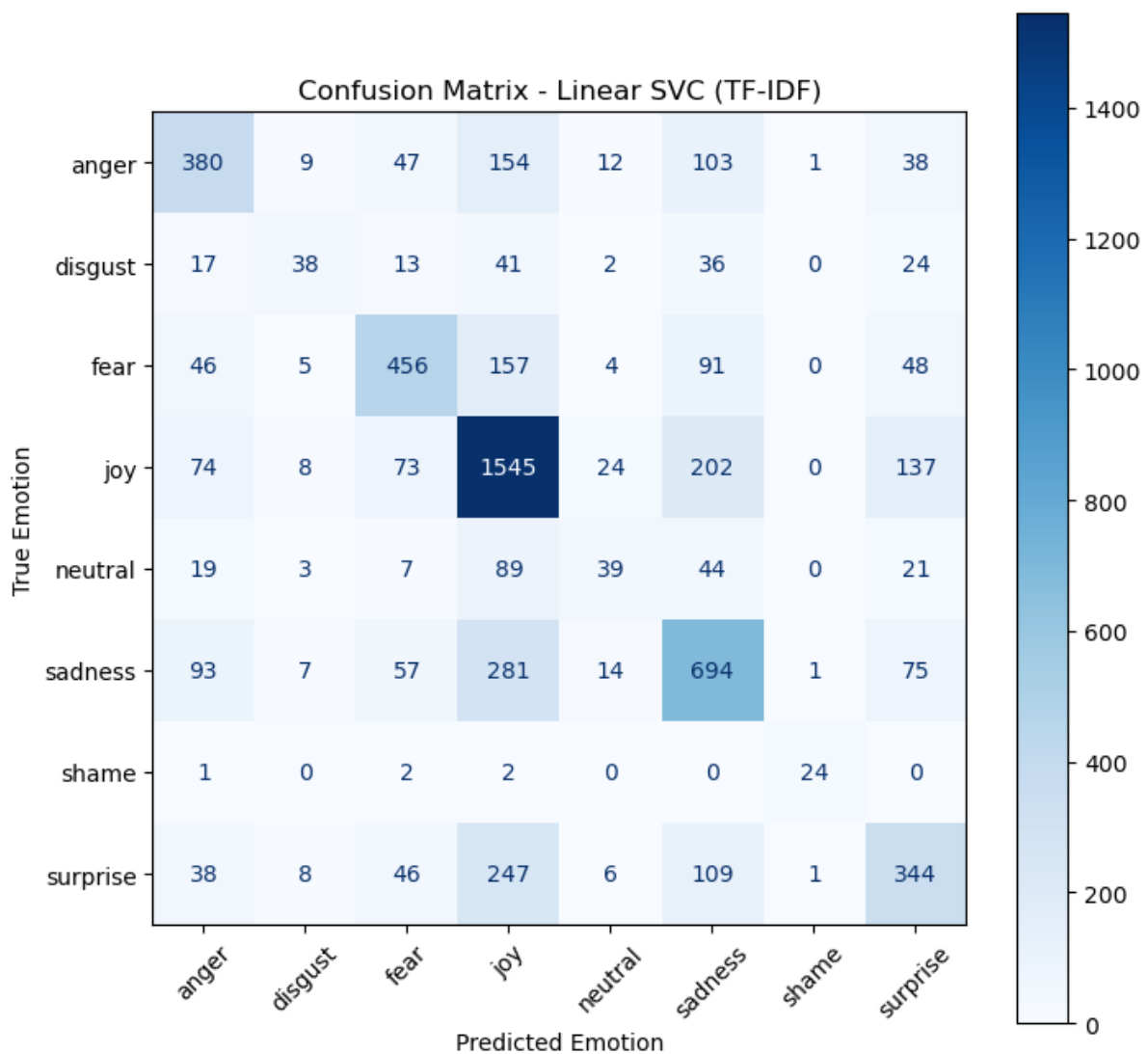
```
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)
```

```
plt.title("Confusion Matrix - Linear SVC (TF-IDF)")
```

```
plt.show()
```

Output:





Code description: Two confusion matrices are built and plotted, one for each representation, using the same sorted list of emotion labels for both.

Logistic Regression - Dataset 2

Task 66: Splitting Dataset 2 into training and testing sets (Logistic Regression)

Description: The cleaned Dataset 2 was split into 80 percent training data and 20 percent testing data for Logistic Regression.

Code:

```
x = copy_d2["sentence"]
```

```
y = copy_d2["emotion"]
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42,
stratify=y)
```

Code description: The same stratified 80:20 split procedure was applied to Dataset 2 for Logistic Regression.

Task 67: Bag of Words vectorization - Dataset 2 (Logistic Regression)

Description: The training and testing text of Dataset 2 was converted into Bag of Words feature vectors.

Code:

```
bow = CountVectorizer()
x_train_bow = bow.fit_transform(x_train)
x_test_bow = bow.transform(x_test)
```

Code description: The vectorizer is fit only on the training text of Dataset 2.

Task 68: TF-IDF vectorization - Dataset 2 (Logistic Regression)

Description: The training and testing text of Dataset 2 was converted into TF-IDF feature vectors.

Code:

```
tfidf = TfidfVectorizer()
x_train_tfidf = tfidf.fit_transform(x_train)
x_test_tfidf = tfidf.transform(x_test)
```

Code description: The TF-IDF vectorizer is fit only on the training text of Dataset 2, mirroring Dataset 1.

Task 69: Training the Logistic Regression model - Dataset 2

Description: A LogisticRegression model was created and fit, first on the Bag of Words vectors and then on the TF-IDF vectors.

Code:

```
lr = LogisticRegression(max_iter=1000, random_state=42)
```

```
lr.fit(x_train_bow, y_train)
```

```
lr.fit(x_train_tfidf, y_train)
```

Output:

```
LogisticRegression(max_iter=1000, random_state=42)
```

Code description: LogisticRegression(max_iter=1000, random_state=42) creates the classifier with a raised iteration limit and a fixed random state, and fit trains it on the given feature vectors together with their corresponding labels.

Task 70: Predicting with the Logistic Regression model - Dataset 2

Description: The fitted model was used to predict the emotion label of the test documents under both representations.

Code:

```
y_pred_bow = lr.predict(x_test_bow)
y_pred_tfidf = lr.predict(x_test_tfidf)
```

Code description: predict assigns an emotion label to each test document, once using the Bag of Words vectors and once using the TF-IDF vectors.

Task 71: Evaluation of the Bag of Words and TF-IDF results - Dataset 2 (Logistic Regression)

Description: Both results for Dataset 2 were scored on the test set using accuracy, precision, recall, and F1.

Code:

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)
precision_bow = precision_score(y_test, y_pred_bow, average = "weighted")
precision_tfidf = precision_score(y_test, y_pred_tfidf, average = "weighted")
recall_bow = recall_score(y_test, y_pred_bow, average = "weighted")
recall_tfidf = recall_score(y_test, y_pred_tfidf, average = "weighted")
f1_bow = f1_score(y_test, y_pred_bow, average = "weighted")
f1_tfidf = f1_score(y_test, y_pred_tfidf, average = "weighted")

print("Using Bag of Words:")
print(f'Accuracy:{accuracy_bow:.2%}')
print(f'Precision:{precision_bow:.2%}')
print(f'Recall:{recall_bow:.2%}')
print(f'F1-Score:{f1_bow:.2%}')
```

```

print("\nUsing TF-IDF:")
print(f'Accuracy:{accuracy_tfidf:.2%}')
print(f'Precision:{precision_tfidf:.2%}')
print(f'Recall:{recall_tfidf:.2%}')
print(f'F1-Score:{f1_tfidf:.2%}')

```

Output:

Using Bag of Words:

Accuracy:90.42%

Precision:90.40%

Recall:90.42%

F1-Score:90.40%

Using TF-IDF:

Accuracy:90.32%

Precision:90.25%

Recall:90.32%

F1-Score:90.26%

Code description: Each metric is computed once for the Bag of Words predictions and once for the TF-IDF predictions, then printed with percentage formatting.

Task 72: Confusion matrix for the Bag of Words and TF-IDF results - Dataset 2 (Logistic Regression)

Description: A confusion matrix was generated for each representation on Dataset 2.

Code:

```

emo_labels = sorted(copy_d2["emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Logistic Regression (Bag of words)")
plt.show()

```

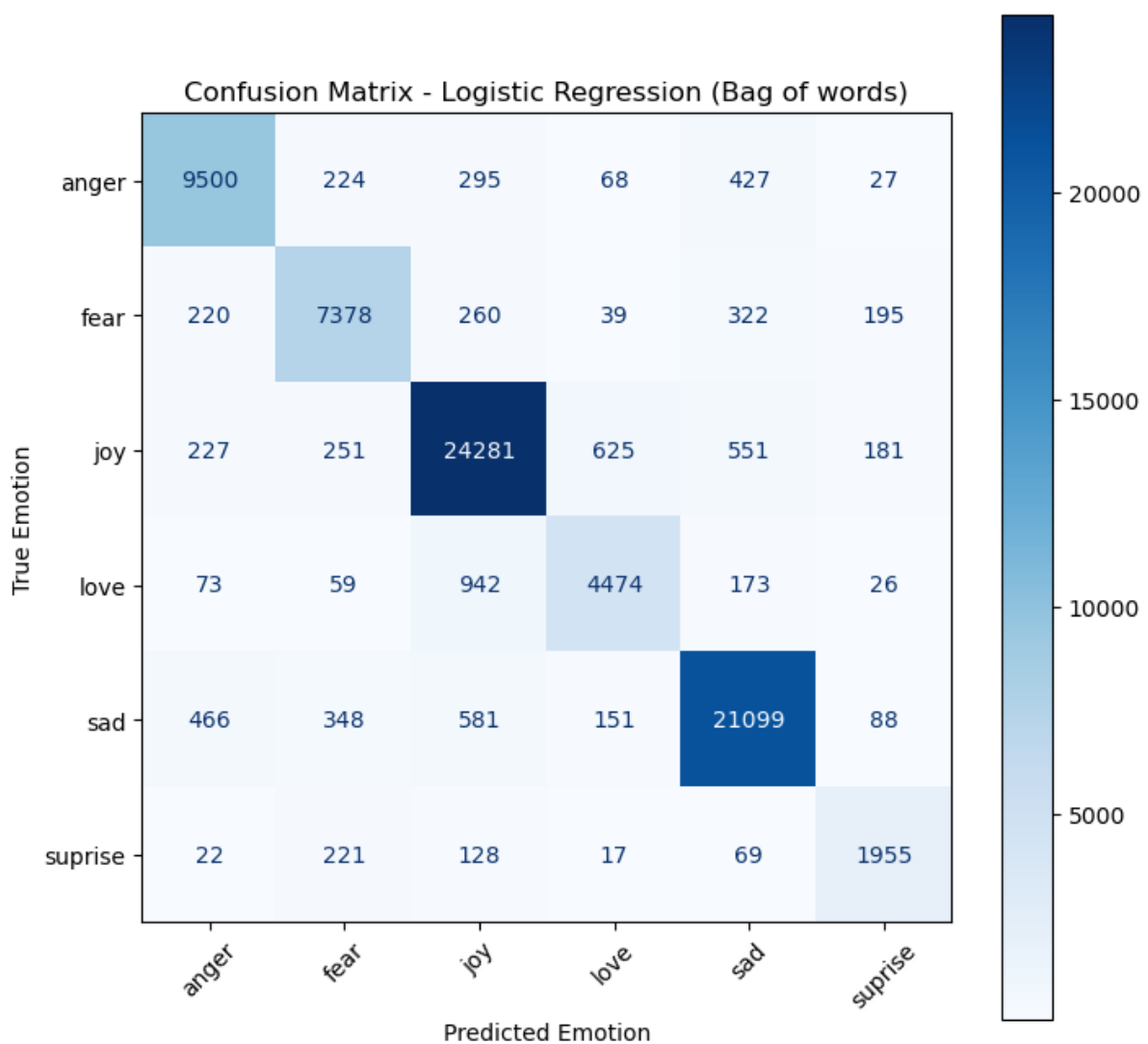
```

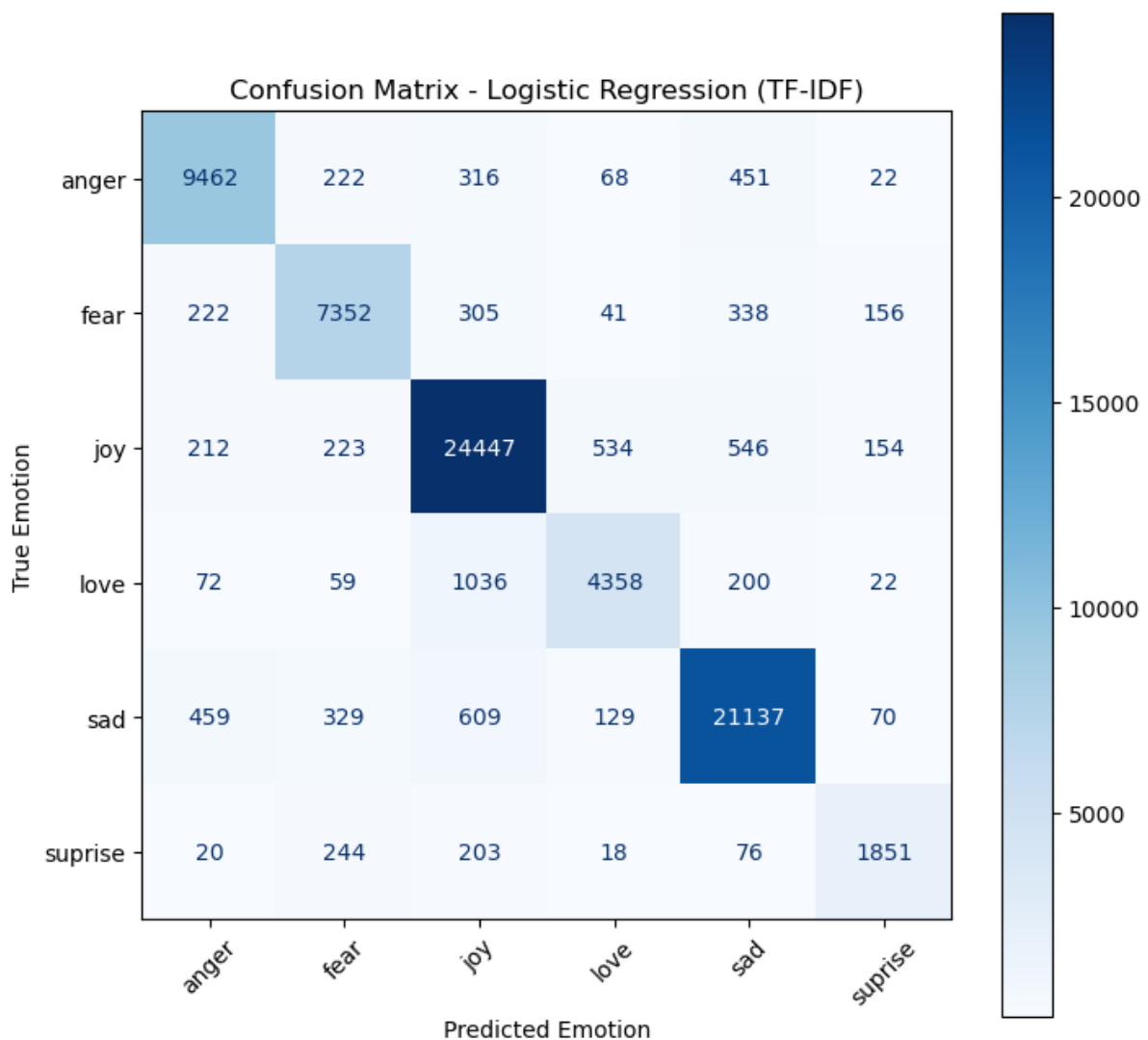
cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)

fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix - Logistic Regression (TF-IDF)")
plt.show()

```

Output:





Code description: Two confusion matrices are built and plotted, one for each representation, using the same sorted list of emotion labels for both.

Support Vector Machine (Linear SVC) - Dataset 2

Task 73: Splitting Dataset 2 into training and testing sets (Support Vector Machine)

Description: The cleaned Dataset 2 was split into 80 percent training data and 20 percent testing data for the Support Vector Machine.

Code:

```
X_2 = copy_d2['sentence']
y_2 = copy_d2['emotion']
```

```
X2_train, X2_test, y2_train, y2_test = train_test_split(X_2, y_2, test_size=0.2,
random_state=42, stratify=y_2)
```

Code description: The same stratified 80:20 split procedure used throughout the project was applied here for the Support Vector Machine.

Task 74: Bag of Words vectorization, training, and evaluation - Dataset 2 (Support Vector Machine)

Description: The training and testing text of Dataset 2 was converted into Bag of Words vectors, a LinearSVC model was trained on them, and the result was evaluated with accuracy, precision, recall, and F1.

Code:

```
bow_2 = CountVectorizer()
X2_train_bow = bow_2.fit_transform(X2_train)
X2_test_bow = bow_2.transform(X2_test)

svm_2_bow = LinearSVC()
svm_2_bow.fit(X2_train_bow, y2_train)
pred_2_bow = svm_2_bow.predict(X2_test_bow)

print("Accuracy: ", accuracy_score(y2_test, pred_2_bow))
print("Precision: ", precision_score(y2_test, pred_2_bow, average='weighted'))
print("Recall: ", recall_score(y2_test, pred_2_bow, average='weighted'))
print("F1 Score: ", f1_score(y2_test, pred_2_bow, average='weighted'))
```

Output:

```
Accuracy: 0.9037294472308888
Precision: 0.9034395340252249
Recall: 0.9037294472308888
F1 Score: 0.9034708529643645
```

Code description: CountVectorizer builds the Bag of Words features, LinearSVC() creates the classifier with its default configuration, fit trains it, predict labels the test documents, and the four metric functions score the predictions using weighted averaging.

Task 75: Confusion matrix for the Bag of Words model - Dataset 2 (Support Vector Machine)

Description: A confusion matrix was generated for the Bag of Words Support Vector Machine model.

Code:

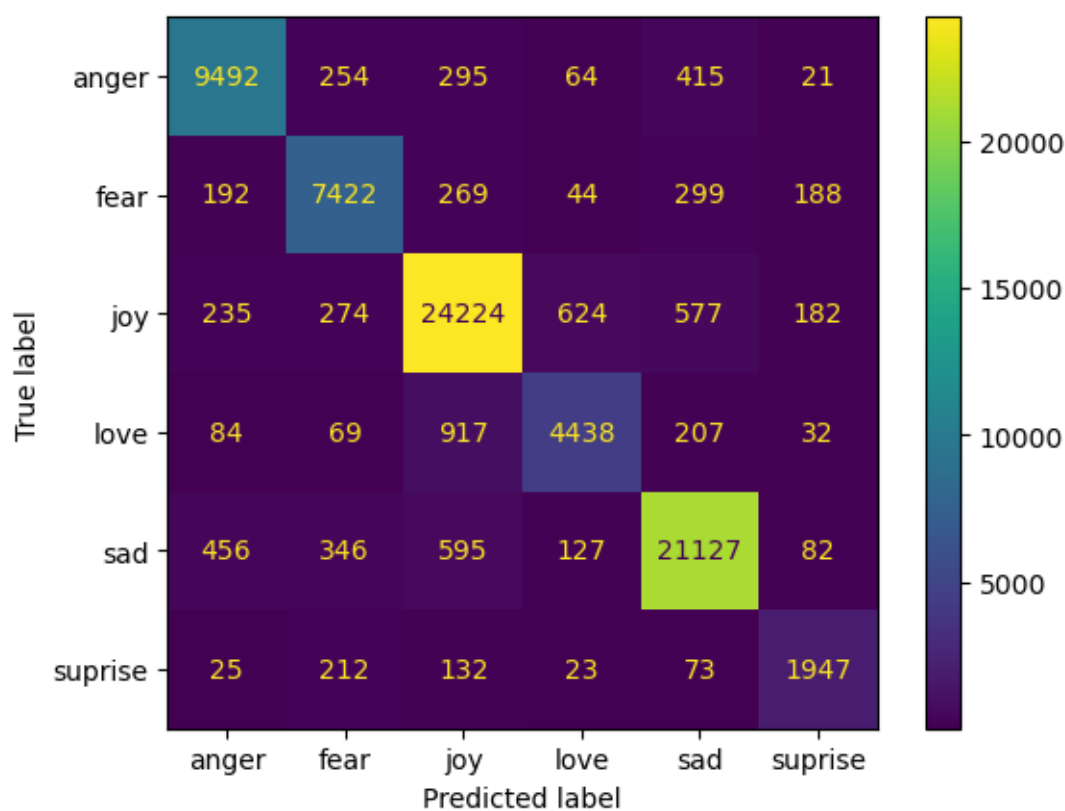
```
cm_2_bow = confusion_matrix(y2_test, pred_2_bow)

disp_2_bow = ConfusionMatrixDisplay(confusion_matrix=cm_2_bow,
display_labels=svm_2_bow.classes_)

disp_2_bow.plot()

plt.show()
```

Output:



Code description: confusion_matrix and ConfusionMatrixDisplay build and render the matrix, using the class labels stored on the fitted model, svm_2_bow.classes_, to order the axes.

Task 76: TF-IDF vectorization, training, and evaluation - Dataset 2 (Support Vector Machine)

Description: The training and testing text of Dataset 2 was converted into TF-IDF vectors, a separate LinearSVC model was trained on them, and the result was evaluated with the same four metrics.

Code:

```

tfidf_2 = TfidfVectorizer()
X2_train_tfidf = tfidf_2.fit_transform(X2_train)
X2_test_tfidf = tfidf_2.transform(X2_test)

svm_2_tfidf = LinearSVC()
svm_2_tfidf.fit(X2_train_tfidf, y2_train)
pred_2_tfidf = svm_2_tfidf.predict(X2_test_tfidf)

print("Accuracy: ", accuracy_score(y2_test, pred_2_tfidf))
print("Precision: ", precision_score(y2_test, pred_2_tfidf, average='weighted'))
print("Recall: ", recall_score(y2_test, pred_2_tfidf, average='weighted'))
print("F1 Score: ", f1_score(y2_test, pred_2_tfidf, average='weighted'))

```

Output:

Accuracy: 0.906941537327383

Precision: 0.9064632931987588

Recall: 0.906941537327383

F1 Score: 0.906499234000669

Code description: TfidfVectorizer builds the TF-IDF features, LinearSVC() creates a separate classifier instance, fit trains it, predict labels the test documents, and the four metric functions score the predictions using weighted averaging.

Task 77: Confusion matrix for the TF-IDF model - Dataset 2 (Support Vector Machine)

Description: A confusion matrix was generated for the TF-IDF Support Vector Machine model.

Code:

```

cm_2_tfidf = confusion_matrix(y2_test, pred_2_tfidf)

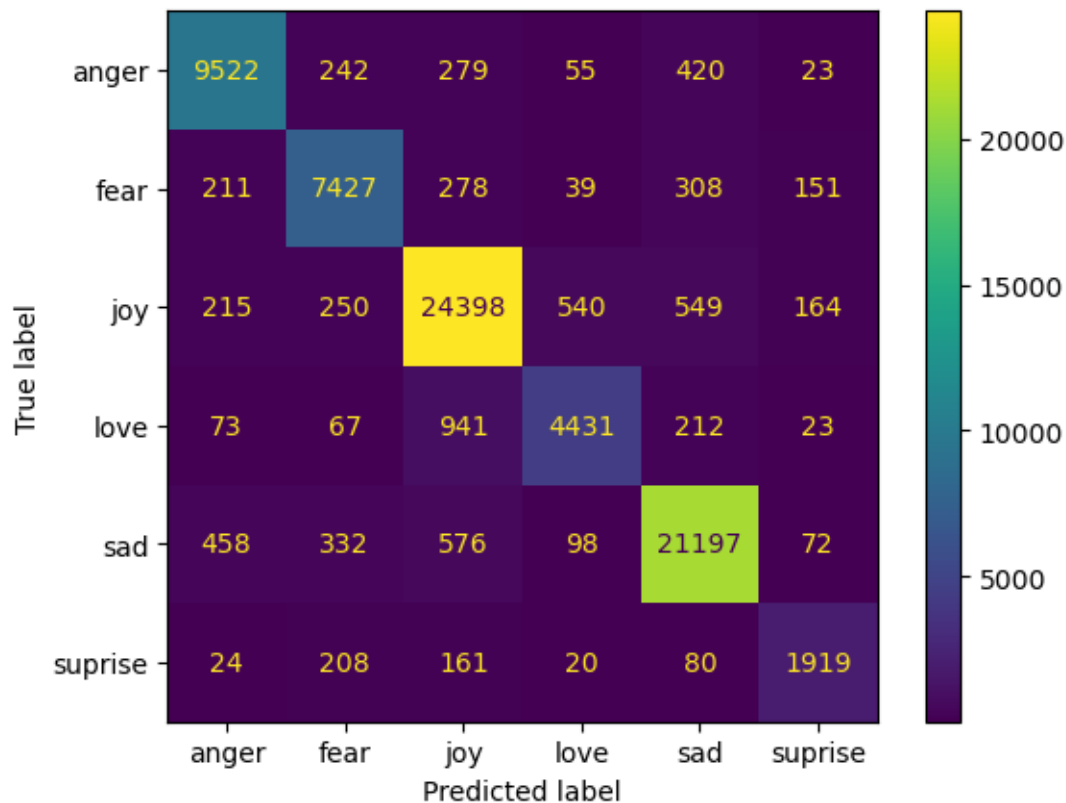
disp_2_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_2_tfidf,
display_labels=svm_2_tfidf.classes_)

disp_2_tfidf.plot()

plt.show()

```

Output:



Code description: confusion_matrix and ConfusionMatrixDisplay build and render the matrix, using the class labels stored on the fitted model, svm_2_tfidf.classes_, to order the axes.

Result Table Discussion

Algorithm	Representation	Dataset 1	Dataset 2
Naive Bayes	BoW	Accuracy: 55.64% Precision: 58.49% Recall: 55.64% F1 Score: 52.13%	Accuracy:85.89% Precision:86.13% Recall:85.89% F1-Score:85.25%
	TF-IDF	Accuracy: 47.04% Precision: 58.20% Recall: 47.04% F1 Score: 40.20%	Accuracy:75.84% Precision:80.76% Recall:75.84% F1-Score:72.59%
Logistic Regression	BoW	Accuracy:58.40% Precision:57.88% Recall:58.40% F1-Score:57.33%	Accuracy:90.42% Precision:90.40% Recall:90.42% F1-Score:90.40%
	TF-IDF	Accuracy:58.28% Precision:59.29% Recall:58.28% F1-Score:55.94%	Accuracy:90.32% Precision:90.25% Recall:90.32% F1-Score:90.26%

Support Vector Machine	BoW	Accuracy:56.13% Precision:55.74% Recall:56.13% F1-Score:55.73%	Accuracy: 90.37% Precision: 90.34% Recall: 90.37% F1 Score: 90.35%
	TF-IDF	Accuracy:58.11% Precision:57.34% Recall:58.11% F1-Score:57.11%	Accuracy: 90.69% Precision: 90.65% Recall: 90.69% F1 Score: 90.65%

The table above brings together all three algorithms across both representations and both datasets, with the full precision, recall, and F1 values given under each evaluation task earlier in this report. Dataset 2 scores far higher than Dataset 1 for every algorithm and every representation. Dataset 2 has 379,611 cleaned rows and only six emotion classes, while Dataset 1 has 30,283 rows spread across eight classes, some of which, such as shame and disgust, have only a few hundred examples for a model to learn from.

Logistic Regression and the Support Vector Machine both score slightly higher with TF-IDF than with Bag of Words on both datasets, since down-weighting very common words lets these models lean on more informative terms. Naive Bayes shows the opposite pattern on Dataset 2, where Bag of Words outperforms TF-IDF by a wide margin. This happens because Multinomial Naive Bayes is built around raw word counts, so scaling those counts into fractional TF-IDF weights works against its underlying probability assumptions, and the model becomes biased toward the majority class, joy, at the cost of recall on smaller classes such as surprise and love, as seen directly in the Naive Bayes classification report and confusion matrices.

Overall, Logistic Regression and the Support Vector Machine give the most balanced results across both datasets, with TF-IDF as the stronger representation for both. Naive Bayes is competitive on Dataset 2 with Bag of Words specifically, but its bias toward the majority class under TF-IDF, and its weaker performance on the smaller, more imbalanced Dataset 1, make it the least consistent of the three algorithms in this project.

Project Code

Give the entire code of your project here.

Importing All Necessary Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

Importing Datasets

```
dataset_1 = pd.read_csv("emotion_dataset_raw.csv")
dataset_2 = pd.read_csv("combined_emotion.csv")

print(dataset_1.shape)
print(dataset_2.shape)

(34792, 2)
(422746, 2)

print(len(dataset_1))
print(len(dataset_2))

34792
422746
```

Importing Libraries to preprocess data

```
import re
import string
import nltk

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

nltk.download('stopwords')
nltk.download('punkt')

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\moham\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\moham\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

True

Dataset_1

```
copy_d1 = dataset_1.copy()
```

```
duplicates = copy_d1['Text'].duplicated()
print(duplicates)
```

```
0      False
1      False
2      False
3      False
4      False
```

```
...
34787   False
34788   False
34789   False
34790   False
34791   False
```

Name: Text, Length: 34792, dtype: bool

```
duplicates = copy_d1['Text'].duplicated().sum()
print(duplicates)
```

3682

```
copy_d1 = copy_d1.drop_duplicates(subset=['Text'])
print(copy_d1)
```

	Emotion	Text
0	neutral	Why ?
1	joy	Sage Act upgrade on my to do list for tommorow.
2	sadness	ON THE WAY TO MY HOMEGIRL BABY FUNERAL!!! MAN ...
3	joy	Such an eye ! The true hazel eye-and so brill...
4	joy	@Iluvtriasantos ugh babe.. hugggz for u .! b...
...	...	...
34787	surprise	@MichelGW have you gift! Hope you like it! It'...
34788	joy	The world didnt give it to me..so the world MO...
34789	anger	A man robbed me today .
34790	fear	Youu call it JEALOUSY, I call it of #Losing YO...
34791	sadness	I think about you baby, and I dream about you ...

[31110 rows x 2 columns]

```
has_nan = copy_d1[['Text', 'Emotion']].isna().any()
print(has_nan)
```

```
Text      False
Emotion   False
dtype: bool
```

```
mask = copy_d1['Text'].str.contains(r'href|http|https|www|html|amp|
nbsp', case=False, regex=True, na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'@\w+', case=False, regex=True,
na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'#\w+', case=False, regex=True,
na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'&\w+', case=False, regex=True,
na=False).sum()
print(mask)
mask = ~copy_d1['Text'].str.isascii()
print(mask.sum())

1454
5068
4267
2119
1421
```

```
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
HTML_TAGS = re.compile(r'\b(href|http|https|www|html|amp|nbsp)\w*\b')
```

```
def preprocess_1(text):
    text = text.lower()

    text = re.sub(r'#\w+', '', text)

    text = re.sub(r'@\w+', '', text)

    text = re.sub(r'&\w+', '', text)

    text = HTML_TAGS.sub(' ', text)

    text = text.encode('ascii', 'ignore').decode('ascii')
```



```

text = text.translate(str.maketrans('', '', string.punctuation))

text = re.sub(r'\d+', '', text)

text = re.sub(r'\s+', ' ', text).strip()

tokens = word_tokenize(text)

tokens = [t for t in tokens if t not in stop_words]

tokens = [stemmer.stem(t) for t in tokens]

return ' '.join(tokens)

nltk.download('punkt_tab')
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\moham\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!

True

copy_d1['Text'] = copy_d1['Text'].apply(preprocess_1)
print("done")

done

mask = copy_d1['Text'].str.contains(r'href|http|https|www|html|amp|
nbsp', case=False, regex=True, na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'@\w+', case=False, regex=True,
na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'#\w+', case=False, regex=True,
na=False).sum()
print(mask)
mask = copy_d1['Text'].str.contains(r'&\w+;', case=False, regex=True,
na=False).sum()
print(mask)
mask = ~copy_d1['Text'].str.isascii()
print(mask.sum())

316
0
0

```

```

0
0

print(copy_d1['Text'].isnull().sum())

print((copy_d1['Text'] == '').sum())

print(copy_d1[copy_d1['Text'] == ''])

copy_d1 = copy_d1[copy_d1['Text'] != '']
copy_d1 = copy_d1.reset_index(drop=True)

0
149
      Emotion Text
0      neutral
76      neutral
114      neutral
269      neutral
392      neutral
...      ...
34000      neutral
34234      neutral
34243      neutral
34429      neutral
34662      neutral

[149 rows x 2 columns]

print((copy_d1['Text'] == '').sum())

0

mask = copy_d1['Text'].str.contains(r'href|http|https|www|html|amp|
nbsp', case=False, regex=True, na=False).sum()
print(mask)

316

print("Original Dataset: ", dataset_1.head(10))
print("Processed Dataset: ", copy_d1.head(10))

Original Dataset:      Emotion
Text
0      neutral
1      joy      Sage Act upgrade on my to do list for tommorow.
2      sadness  ON THE WAY TO MY HOMEGIRL BABY FUNERAL!!! MAN ...
3      joy      Such an eye ! The true hazel eye-and so brill...
4      joy      @Iluvtriasantos ugh babe.. hugggz for u .! b...
5      fear     I'm expecting an extremely important phonecall...
Why ?

```

```

6    sadness    .Couldnt wait to see them live. If missing th...
7    surprise   maken Tip 2: Stop op een moment dat je het hel...
8    surprise   En dan krijg je ff een cadeautje van een tweep...
9    surprise   @1116am Drummer Boy bij op verzoek van @Biem0...
Processed Dataset:      Emotion
Text
0      joy                sage act upgrad list tomorrow
1    sadness   way homegirl babi funer man hate funer realli ...
2      joy     eye true hazel eyeand brilliant regular featur...
3      joy     ugh babe hugggzzz u babe naamaz nga ako e babe...
4      fear                im expect extrem import phonecal minut
5    sadness   couldnt wait see live miss nh wasnt pain enuf ...
6    surprise   maken tip stop op een moment dat je het hele p...
7    surprise           en dan krijg je ff een cadeautj van een tweep
8    surprise   drummer boy bij op verzoek van welk uitvoer va...
9      anger                bull toss effigi hand becam infuri

print(copy_d1.shape)

(30961, 2)

duplicates = copy_d1['Text'].duplicated().sum()
print(duplicates)

678

copy_d1 = copy_d1.drop_duplicates(subset=['Text'])

duplicates = copy_d1['Text'].duplicated().sum()
print(duplicates)

0

print(copy_d1.shape)

(30283, 2)

```

Dataset_2

```

copy_d2 = dataset_2.copy()
print(copy_d2)

```

	sentence	emotion
0	i just feel really helpless and heavy hearted	fear
1	ive enjoyed being able to slouch about relax a...	sad
2	i gave up my internship with the dmrg and am f...	fear
3	i dont know i feel so lost	sad
4	i am a kindergarten teacher and i am thoroughl...	fear
...	...	...
422741	i begun to feel distressed for you	fear

422742	i left feeling annoyed and angry thinking that...	anger
422743	i were to ever get married i d have everything...	joy
422744	i feel reluctant in applying there because i w...	fear
422745	i just wanted to apologize to you because i fe...	anger

[422746 rows x 2 columns]

```
duplicates = copy_d2['sentence'].duplicated()
print(duplicates)
```

```
0      False
1      False
2      False
3      False
4      False
```

```
...
422741    True
422742    True
422743    True
422744    True
422745    True
```

Name: sentence, Length: 422746, dtype: bool

```
duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)
```

28924

```
copy_d2 = copy_d2.drop_duplicates(subset=['sentence'])
```

```
has_nan = copy_d2[['sentence', 'emotion']].isna().any()
print(has_nan)
```

```
sentence    False
emotion     False
dtype: bool
```

```
mask = copy_d2['sentence'].str.contains(r'href|http|https|www|html|
&nbsp;', case=False, regex=True, na=False).sum()
```

```
print(mask)
```

```
mask = copy_d2['sentence'].str.contains(r'@\w+', case=False,
regex=True, na=False).sum()
```

```
print(mask)
```

```
mask = copy_d2['sentence'].str.contains(r'#\w+', case=False,
regex=True, na=False).sum()
```

```
print(mask)
```

```
mask = copy_d2['sentence'].str.contains(r'&\w+;', case=False,
regex=True, na=False).sum()
```

```
print(mask)
```

```
mask = ~copy_d2['sentence'].str.isascii()
```

```
print(mask.sum())
```

9244

0

0

0

0

```
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
HTML_TAGS = re.compile(r'\b(href|http|https|www|html|amp|nbsp)\w*\b')
```

```
def preprocess_2(text):
```

```
    text = text.lower()
```

```
    text = HTML_TAGS.sub(' ', text)
```

```
    text = text.translate(str.maketrans('', '', string.punctuation))
```

```
    text = re.sub(r'\d+', ' ', text)
```

```
    text = re.sub(r'\s+', ' ', text).strip()
```

```
    tokens = word_tokenize(text)
```

```
    tokens = [t for t in tokens if t not in stop_words]
```

```
    tokens = [stemmer.stem(t) for t in tokens]
```

```
    return ' '.join(tokens)
```

```
copy_d2['sentence'] = copy_d2['sentence'].apply(preprocess_2)
print("done")
```

done

```
mask = copy_d2['sentence'].str.contains(r'href|http|https|www|html|
amp|nbsp', case=False, regex=True, na=False).sum()
print(mask)
```

2008

```

print(copy_d2['sentence'].isnull().sum())
print((copy_d2['sentence'] == '').sum())
0
9

copy_d2 = copy_d2[copy_d2['sentence'] != '']
copy_d2 = copy_d2.reset_index(drop=True)

print((copy_d2['sentence'] == '').sum())
0

print("Original Dataset: " ,dataset_2.head(10))
print("Processed Dataset: " ,copy_d2.head(10))

Original Dataset:
sentence  emotion
0      i just feel really helpless and heavy hearted      fear
1  ive enjoyed being able to slouch about relax a...      sad
2  i gave up my internship with the dmrgr and am f...      fear
3              i dont know i feel so lost      sad
4  i am a kindergarten teacher and i am thoroughl...      fear
5      i was beginning to feel quite disheartened      sad
6  i would think that whomever would be lucky eno...      love
7  i fear that they won t ever feel that deliciou...      joy
8  im forever taking some time out to have a lie ...      surprise
9  i can still lose the weight without feeling de...      sad
Processed Dataset:
sentence  emotion
0      feel realli helpless heavi heart      fear
1  ive enjoy abl slouch relax unwind frankli need...      sad
2      gave internship dmrgr feel distraught      fear
3              dont know feel lost      sad
4  kindergarten teacher thoroughli weari job take...      fear
5              begin feel quit dishearten      sad
6  would think whomev would lucki enough stay sui...      love
7  fear ever feel delici excit christma eve least...      joy
8      im forev take time lie feel weird      surprise
9      still lose weight without feel depriv      sad

print(copy_d2.shape)

(393813, 2)

duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)

14002

```

```

copy_d2 = copy_d2.drop_duplicates(subset=['sentence'])
duplicates = copy_d2['sentence'].duplicated().sum()
print(duplicates)

0

print(copy_d2.shape)

(379811, 2)

```

Nahian - Naive Bayes - Multinomial Naive Bayes

Dataset_1

```

XN_1 = copy_d1['Text']
YN_1 = copy_d1['Emotion']

from sklearn.model_selection import train_test_split
XN1_train, XN1_test, YN1_train, YN1_test = train_test_split(XN_1,
YN_1, test_size=0.2, random_state=42, stratify=YN_1)

```

Bag of Words

```

Bag_o_w = CountVectorizer()
XN_train_bow = Bag_o_w.fit_transform(XN1_train)
XN_test_bow = Bag_o_w.transform(XN1_test)

MNB_BW = MultinomialNB()
MNB_BW.fit(XN_train_bow, YN1_train)

predictions_BW = MNB_BW.predict(XN_test_bow)
print("Accuracy score: ", accuracy_score(YN1_test, predictions_BW))
print("Precision: ", precision_score(YN1_test, predictions_BW,
average='weighted'))
print("Recall: ", recall_score(YN1_test, predictions_BW,
average='weighted'))
print("F1 Score: ", f1_score(YN1_test, predictions_BW,
average='weighted'))

Accuracy score:  0.5563810467228001
Precision:  0.5848819678792966
Recall:  0.5563810467228001
F1 Score:  0.5212632964009482

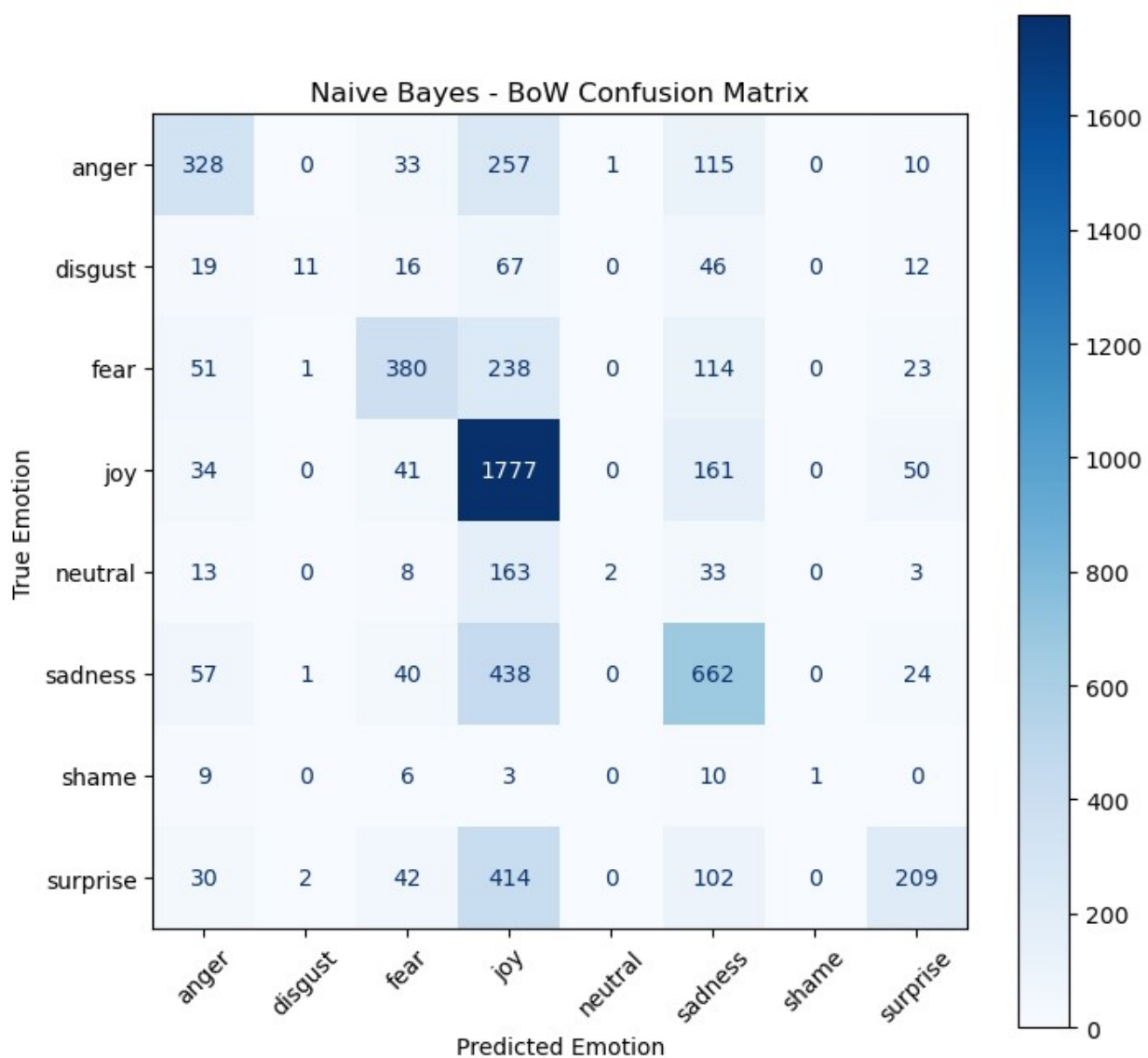
emo_labels = sorted(YN1_test.unique())
NBcm_bow = confusion_matrix(YN1_test, predictions_BW,
labels=emo_labels)
NBcm_display_bow = ConfusionMatrixDisplay(confusion_matrix=NBcm_bow,
display_labels=emo_labels)

```

```

fig, ax = plt.subplots(figsize=(8, 8))
NBcm_display_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Naive Bayes - BoW Confusion Matrix")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()

```



TF-IDF

```

TF_IDF_c = TfidfVectorizer()
XN_train_TFIDF = TF_IDF_c.fit_transform(XN1_train)
XN_test_TFIDF = TF_IDF_c.transform(XN1_test)

```



```

MNB_TFIDF = MultinomialNB()
MNB_TFIDF.fit(XN_train_TFIDF, YN1_train)

predictions_TFIDF = MNB_TFIDF.predict(XN_test_TFIDF)
print("Accuracy score: ", accuracy_score(YN1_test, predictions_TFIDF))
print("Precision: ", precision_score(YN1_test, predictions_TFIDF,
average='weighted'))
print("Recall: ", recall_score(YN1_test, predictions_TFIDF,
average='weighted'))
print("F1 Score: ", f1_score(YN1_test, predictions_TFIDF,
average='weighted'))

Accuracy score:  0.47036486709592207
Precision:  0.5820018828221848
Recall:  0.47036486709592207
F1 Score:  0.4019746575054667

D:\NLP\NLP Project\env\Lib\site-packages\sklearn\metrics\
_classification.py:1833: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])

from collections import Counter

print("Actual classes:")
print(Counter(YN1_test))

print("\nPredicted classes:")
print(Counter(predictions_TFIDF))

Actual classes:
Counter({'joy': 2063, 'sadness': 1222, 'fear': 807, 'surprise': 799,
'anger': 744, 'neutral': 222, 'disgust': 171, 'shame': 29})

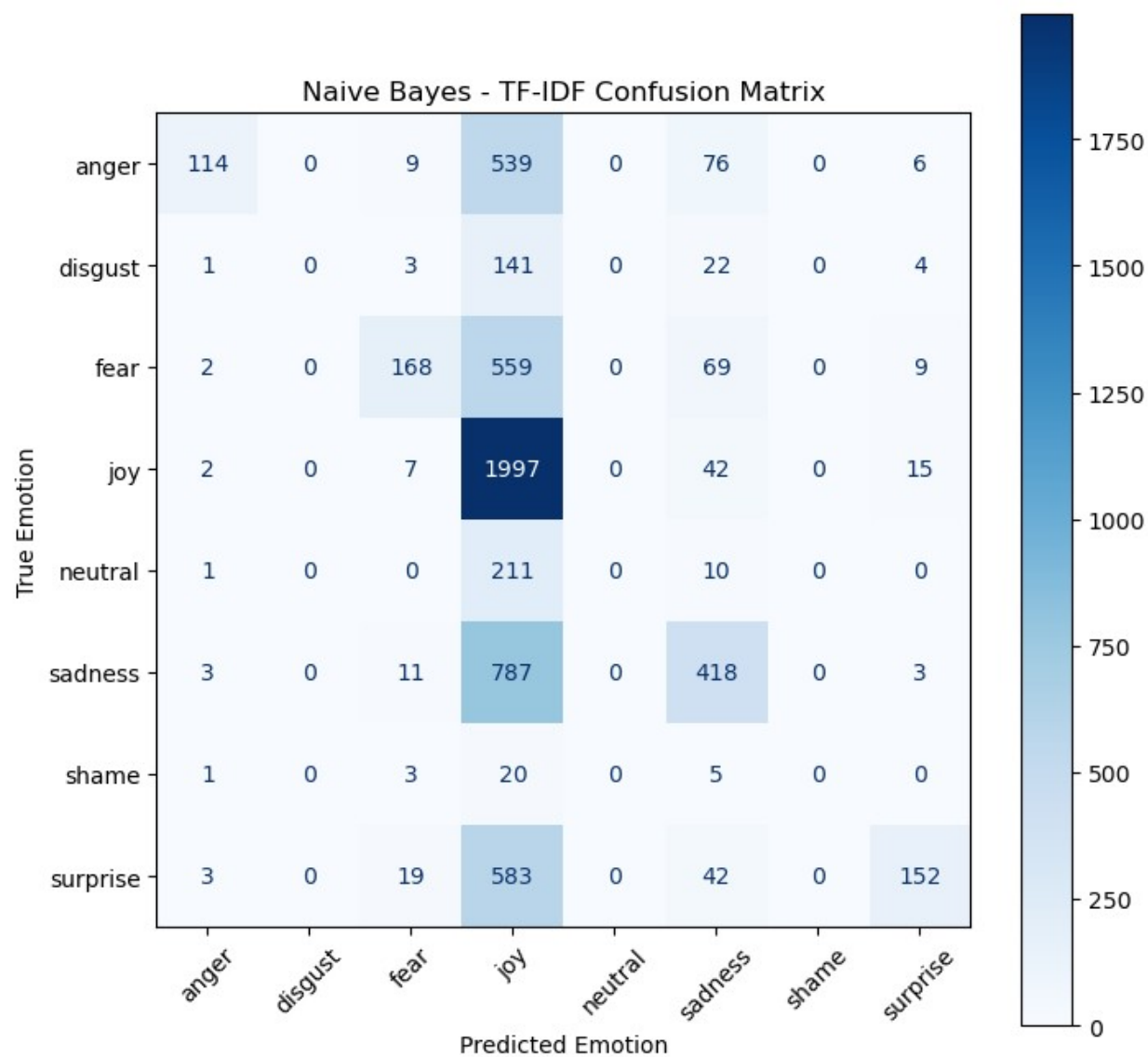
Predicted classes:
Counter({'np.str_('joy)': 4837, np.str_('sadness)': 684,
np.str_('fear)': 220, np.str_('surprise)': 189, np.str_('anger)':
127})

emo_labels = sorted(YN1_test.unique())
NBcm_tfidf = confusion_matrix(YN1_test, predictions_TFIDF,
labels=emo_labels)
NBcm_display_tfidf =
ConfusionMatrixDisplay(confusion_matrix=NBcm_tfidf,
display_labels=emo_labels)

fig, ax = plt.subplots(figsize=(8, 8))
NBcm_display_tfidf.plot(ax=ax, cmap="Blues", xticks_rotation=45)
plt.title("Naive Bayes - TF-IDF Confusion Matrix")

```

```
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```



Tonmoy

Splitting dataset_2 train(80%)-test(20%)

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

xt = copy_d2["sentence"]
yt = copy_d2["emotion"]

xt_train, xt_test, yt_train, yt_test = train_test_split(
    xt,
    yt,
    test_size = 0.2,
    random_state=42,
    stratify=yt
)
```

Bag of Words

```
bow = CountVectorizer()
xt_train_bow = bow.fit_transform(xt_train)
xt_test_bow = bow.transform(xt_test)

MNB_BW = MultinomialNB()
MNB_BW.fit(xt_train_bow, yt_train)

yt_pred_bow = MNB_BW.predict(xt_test_bow)
```

TF-IDF

```
tfidf = TfidfVectorizer()
xt_train_tfidf = tfidf.fit_transform(xt_train)
xt_test_tfidf = tfidf.transform(xt_test)

MNB_TFIDF = MultinomialNB()
MNB_TFIDF.fit(xt_train_tfidf, yt_train)

yt_pred_tfidf = MNB_TFIDF.predict(xt_test_tfidf)
```

Naive Bayes

Evaluation

```
accuracy_bow = accuracy_score(yt_test, yt_pred_bow)
accuracy_tfidf = accuracy_score(yt_test, yt_pred_tfidf)

precision_bow = precision_score(yt_test, yt_pred_bow,
                                average='weighted')
precision_tfidf = precision_score(yt_test, yt_pred_tfidf,
                                   average='weighted')

recall_bow = recall_score(yt_test, yt_pred_bow, average='weighted')
recall_tfidf = recall_score(yt_test, yt_pred_tfidf,
                             average='weighted')

f1_bow = f1_score(yt_test, yt_pred_bow, average='weighted')
f1_tfidf = f1_score(yt_test, yt_pred_tfidf, average='weighted')

print("Using Bag of Words:")
print(f"Accuracy:{accuracy_bow:.2%}")
print(f"Precision:{precision_bow:.2%}")
print(f"Recall:{recall_bow:.2%}")
print(f"F1-Score:{f1_bow:.2%}")

print("\nUsing TF-IDF:")
print(f"Accuracy:{accuracy_tfidf:.2%}")
print(f"Precision:{precision_tfidf:.2%}")
print(f"Recall:{recall_tfidf:.2%}")
print(f"F1-Score:{f1_tfidf:.2%}")
```

Using Bag of Words:

Accuracy:85.89%
Precision:86.13%
Recall:85.89%
F1-Score:85.25%

Using TF-IDF:

Accuracy:75.84%
Precision:80.76%
Recall:75.84%
F1-Score:72.59%

Confusion Matrix

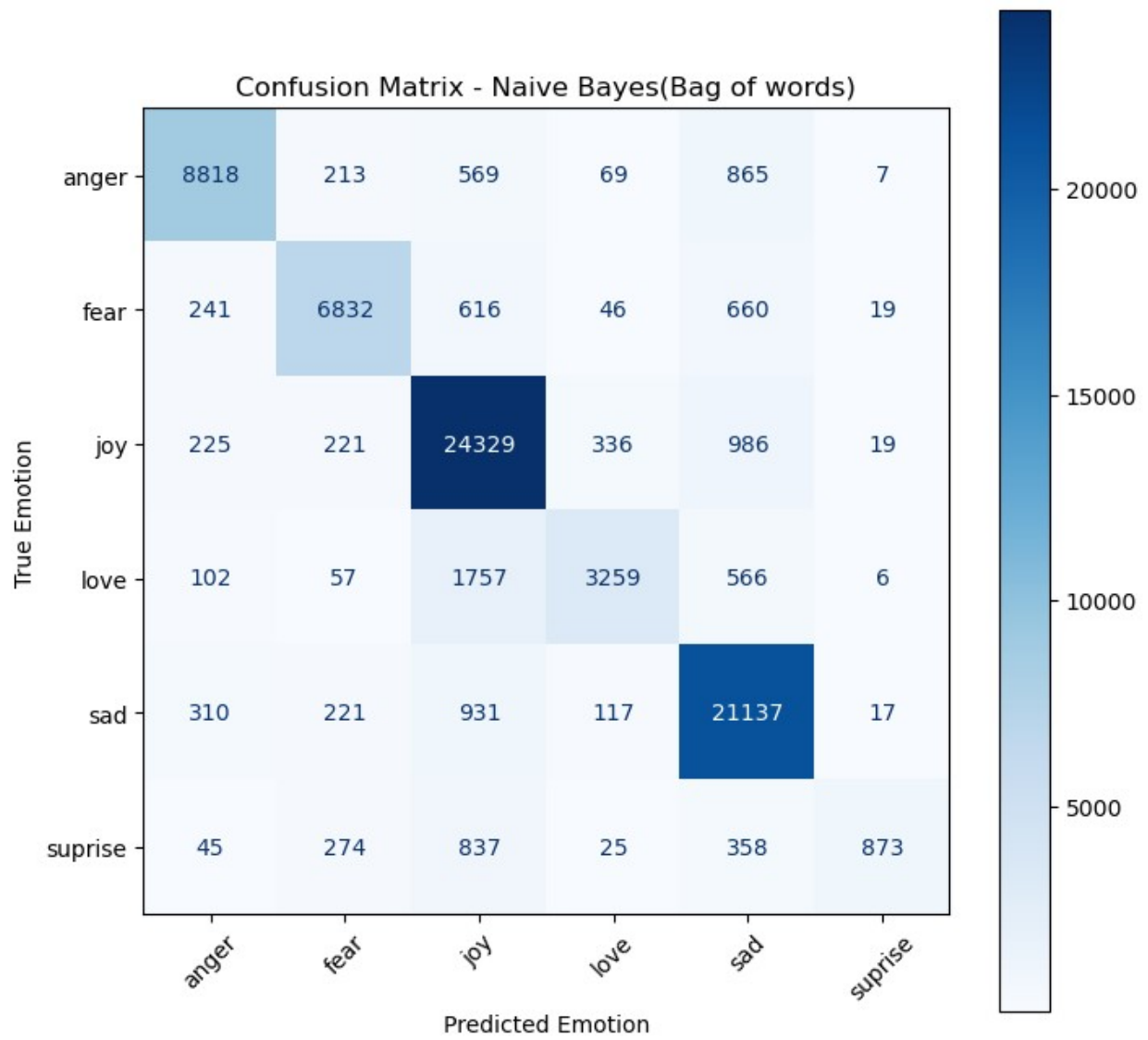
```
emo_labels = sorted(copy_d2["emotion"].unique())
cm_bow = confusion_matrix(yt_test, yt_pred_bow, labels=emo_labels)
```

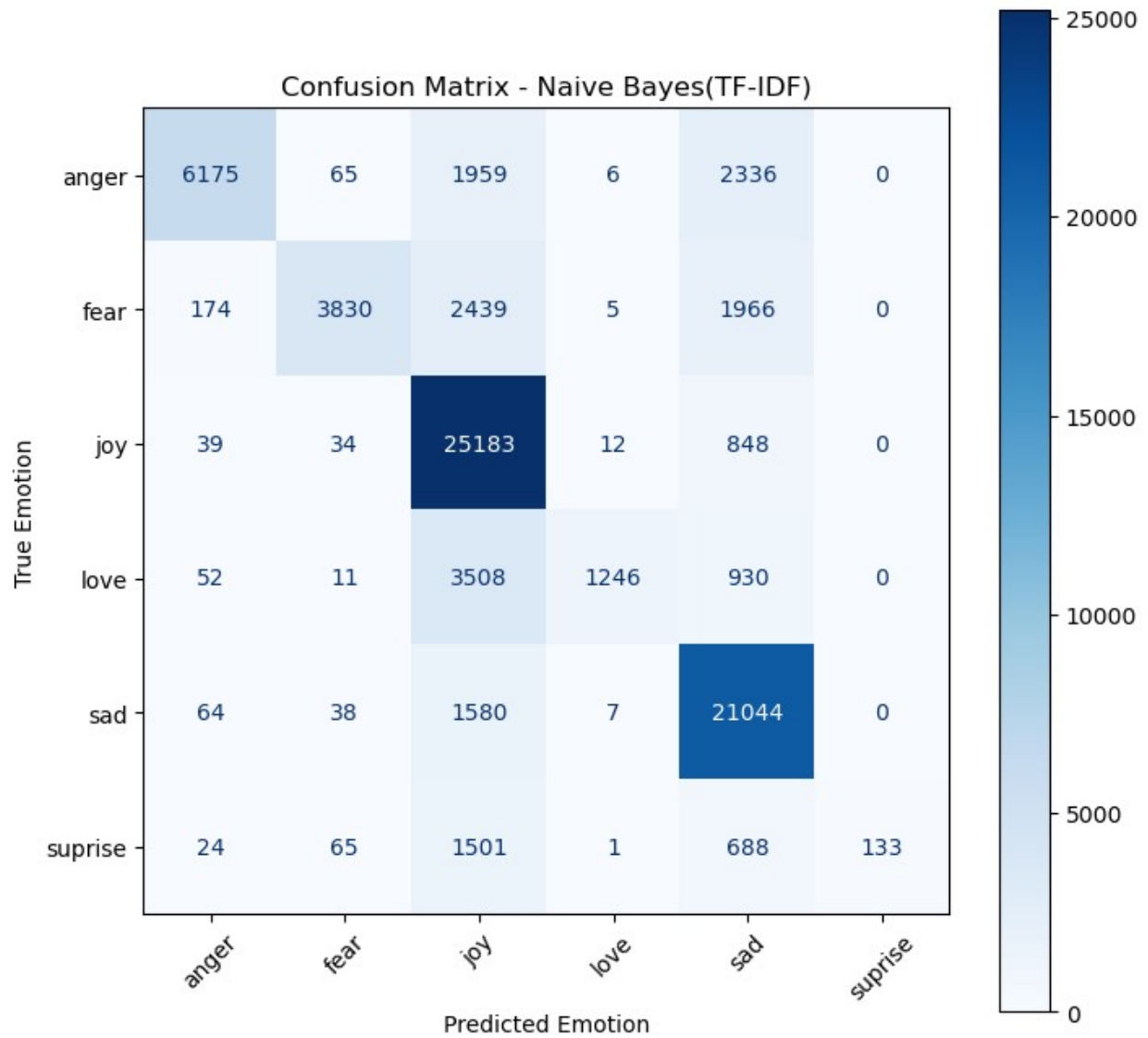
```
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Naive Bayes(Bag of words)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()

cm_tfidf = confusion_matrix(yt_test, yt_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Naive Bayes(TF-IDF)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```





Splitting dataset_1 train(80%)-test(20%)

```
x = copy_d1["Text"]
y = copy_d1["Emotion"]

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Bag of Words

```
bow = CountVectorizer()
x_train_bow = bow.fit_transform(x_train)
x_test_bow = bow.transform(x_test)
```

TF-IDF

```
tfidf = TfidfVectorizer()
x_train_tfidf = tfidf.fit_transform(x_train)
x_test_tfidf = tfidf.transform(x_test)
```

Logistic Regression

```
lr_bow = LogisticRegression(max_iter=1000, random_state=42)
lr_bow.fit(x_train_bow, y_train)

LogisticRegression(max_iter=1000, random_state=42)

lr_tfidf = LogisticRegression(max_iter=1000, random_state=42)
lr_tfidf.fit(x_train_tfidf, y_train)

LogisticRegression(max_iter=1000, random_state=42)

y_pred_bow = lr_bow.predict(x_test_bow)
y_pred_tfidf = lr_tfidf.predict(x_test_tfidf)
```

Evaluation

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)

precision_bow = precision_score(
    y_test,
    y_pred_bow,
    average = "weighted"
)
precision_tfidf = precision_score(
    y_test,
    y_pred_tfidf,
    average = "weighted"
)
```



```

recall_bow = recall_score(
    y_test,
    y_pred_bow,
    average = "weighted"
)
recall_tfidf = recall_score(
    y_test,
    y_pred_tfidf,
    average = "weighted"
)

f1_bow = f1_score(
    y_test,
    y_pred_bow,
    average = "weighted"
)
f1_tfidf = f1_score(
    y_test,
    y_pred_tfidf,
    average = "weighted"
)

print("Using Bag of Words:")
print(f"Accuracy:{accuracy_bow:.2%}")
print(f"Precision:{precision_bow:.2%}")
print(f"Recall:{recall_bow:.2%}")
print(f"F1-Score:{f1_bow:.2%}")
print("\nUsing TF-IDF:")
print(f"Accuracy:{accuracy_tfidf:.2%}")
print(f"Precision:{precision_tfidf:.2%}")
print(f"Recall:{recall_tfidf:.2%}")
print(f"F1-Score:{f1_tfidf:.2%}")

```

Using Bag of Words:
Accuracy:58.40%
Precision:57.88%
Recall:58.40%
F1-Score:57.33%

Using TF-IDF:
Accuracy:58.28%
Precision:59.29%
Recall:58.28%
F1-Score:55.94%

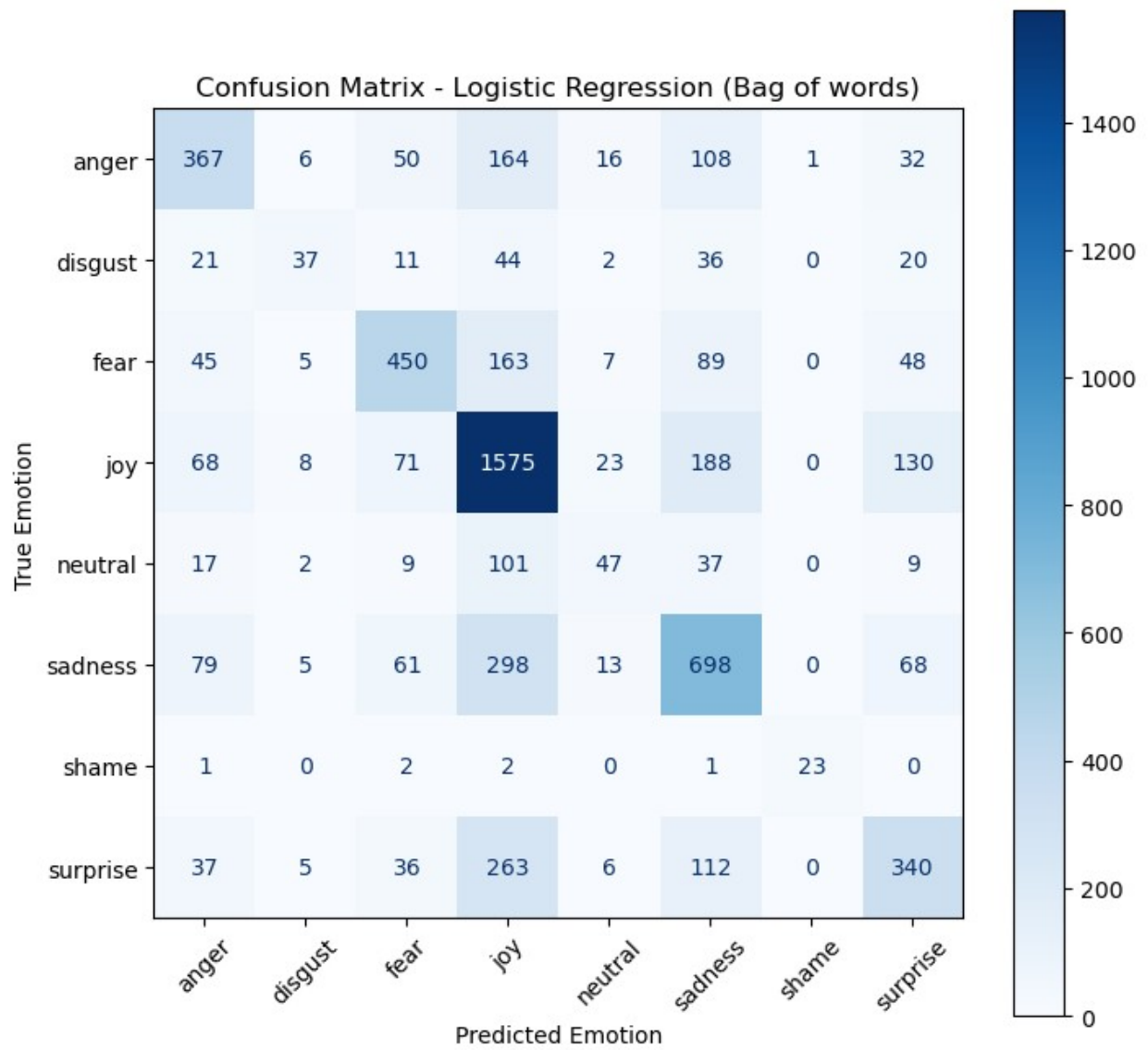
Confusion Matrix

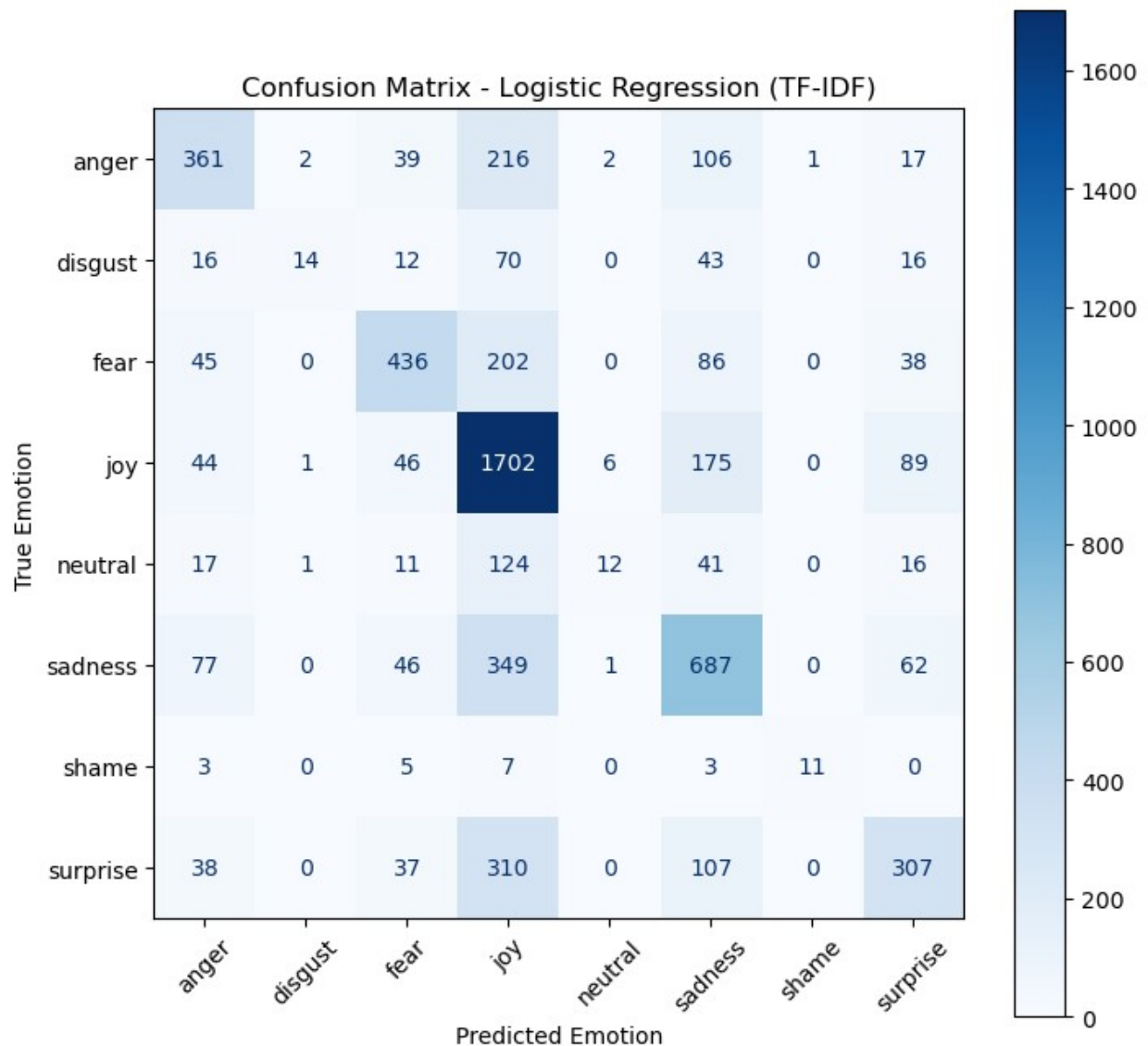
```
emo_labels = sorted(copy_dl["Emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Logistic Regression (Bag of words)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()

cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Logistic Regression (TF-IDF)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```





Samrat

Splitting dataset_1 train(80%)-test(20%)

```
from sklearn.model_selection import train_test_split

x = copy_d1["Text"]
y = copy_d1["Emotion"]

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
```

```
test_size = 0.2,  
random_state=42,  
stratify=y  
)
```

Bag of Words

```
bow = CountVectorizer()  
x_train_bow = bow.fit_transform(x_train)  
x_test_bow = bow.transform(x_test)
```

TF-IDF

```
tfidf = TfidfVectorizer()  
x_train_tfidf = tfidf.fit_transform(x_train)  
x_test_tfidf = tfidf.transform(x_test)
```

Linear Support Vector Classifier(SVC)

```
svc_bow = LinearSVC(random_state=42)  
svc_tfidf = LinearSVC(random_state=42)  
  
svc_bow.fit(x_train_bow, y_train)  
svc_tfidf.fit(x_train_tfidf, y_train)  
  
LinearSVC(random_state=42)  
  
y_pred_bow = svc_bow.predict(x_test_bow)  
y_pred_tfidf = svc_tfidf.predict(x_test_tfidf)
```

Evaluation

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)  
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)  
  
precision_bow = precision_score(  
    y_test,  
    y_pred_bow,  
    average = "weighted"  
)  
precision_tfidf = precision_score(  
    y_test,  
    y_pred_tfidf,
```

```

        average = "weighted"
    )
    recall_bow = recall_score(
        y_test,
        y_pred_bow,
        average = "weighted"
    )
    recall_tfidf = recall_score(
        y_test,
        y_pred_tfidf,
        average = "weighted"
    )

    f1_bow = f1_score(
        y_test,
        y_pred_bow,
        average = "weighted"
    )
    f1_tfidf = f1_score(
        y_test,
        y_pred_tfidf,
        average = "weighted"
    )

    print("Using Bag of Words:")
    print(f"Accuracy:{accuracy_bow:.2%}")
    print(f"Precision:{precision_bow:.2%}")
    print(f"Recall:{recall_bow:.2%}")
    print(f"F1-Score:{f1_bow:.2%}")
    print("\nUsing TF-IDF:")
    print(f"Accuracy:{accuracy_tfidf:.2%}")
    print(f"Precision:{precision_tfidf:.2%}")
    print(f"Recall:{recall_tfidf:.2%}")
    print(f"F1-Score:{f1_tfidf:.2%}")

```

```

Using Bag of Words:
Accuracy:56.13%
Precision:55.74%
Recall:56.13%
F1-Score:55.73%

```

```

Using TF-IDF:
Accuracy:58.11%
Precision:57.34%
Recall:58.11%
F1-Score:57.11%

```

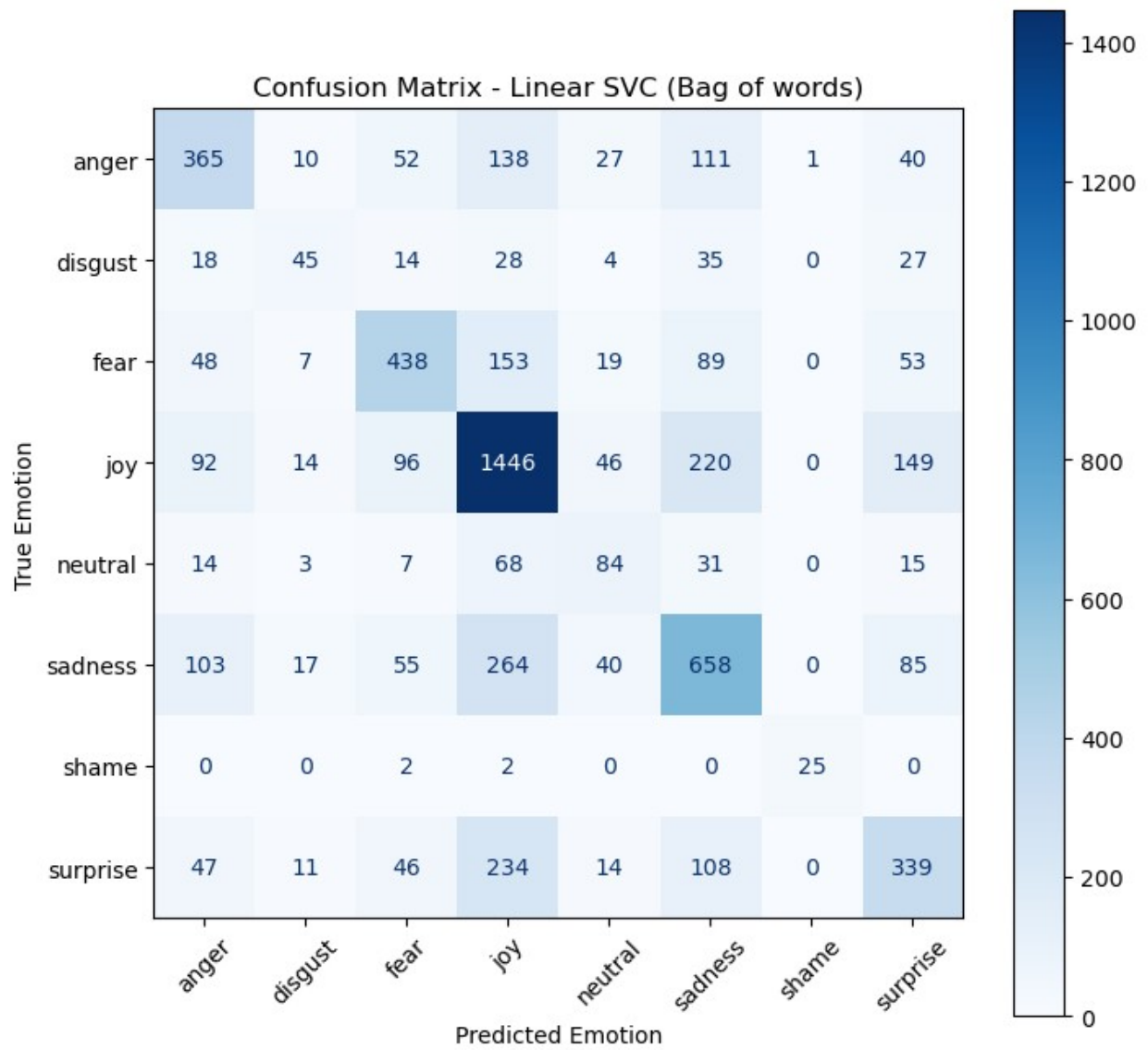
Confusion Matrix

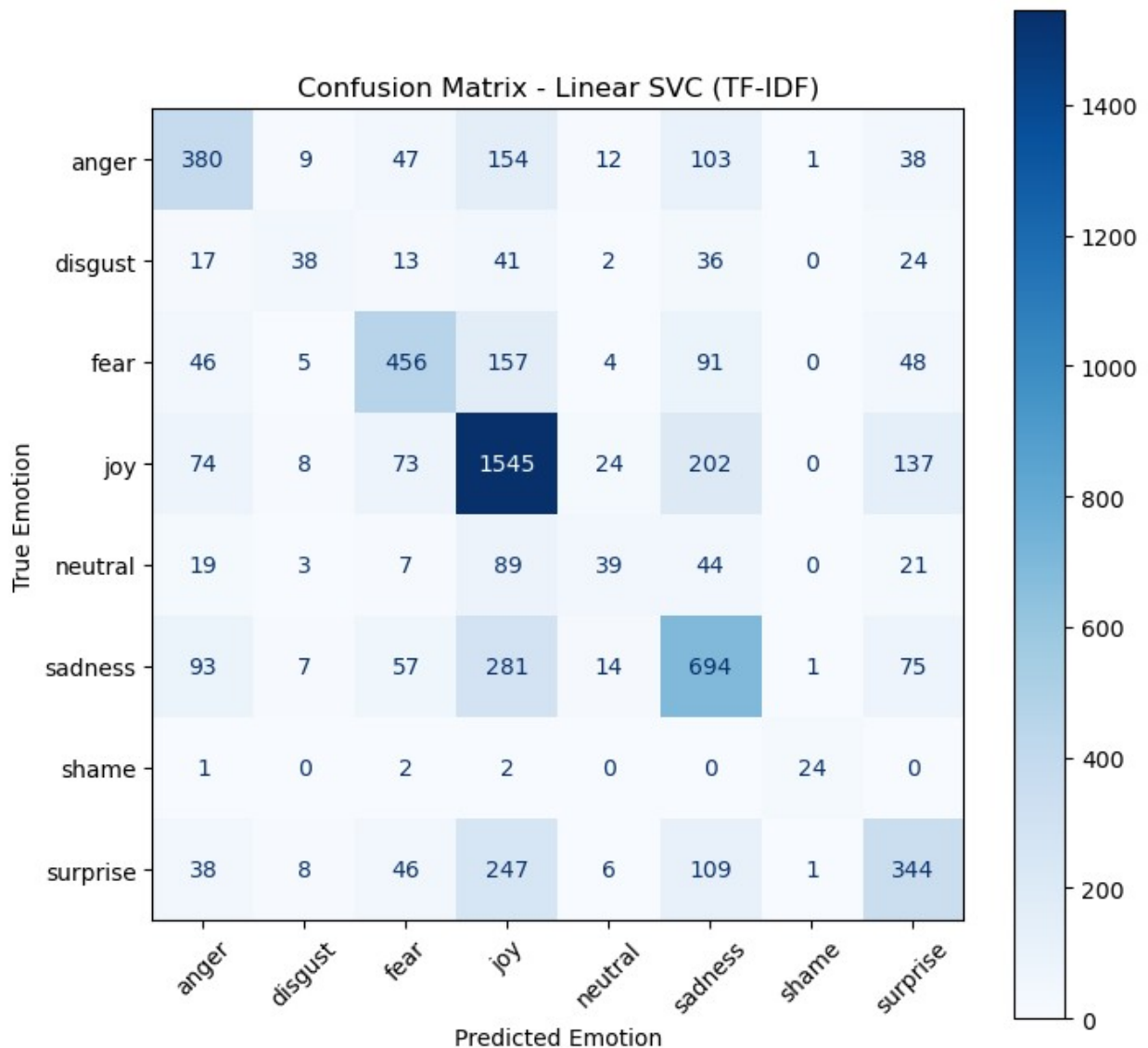
```
emo_labels = sorted(copy_dl["Emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Linear SVC (Bag of words)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()

cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Linear SVC (TF-IDF)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```





Splitting dataset_2 train(80%)-test(20%)

```
x = copy_d2["sentence"]
y = copy_d2["emotion"]

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Bag of Words

```
bow = CountVectorizer()
x_train_bow = bow.fit_transform(x_train)
x_test_bow = bow.transform(x_test)
```

TF-IDF

```
tfidf = TfidfVectorizer()
x_train_tfidf = tfidf.fit_transform(x_train)
x_test_tfidf = tfidf.transform(x_test)
```

Logistic Regression

```
lr_bow = LogisticRegression(
    max_iter=1000,
    random_state=42
)
lr_tfidf = LogisticRegression(
    max_iter=1000,
    random_state=42
)

lr_bow.fit(x_train_bow, y_train)
lr_tfidf.fit(x_train_tfidf, y_train)

LogisticRegression(max_iter=1000, random_state=42)

y_pred_bow = lr_bow.predict(x_test_bow)
y_pred_tfidf = lr_tfidf.predict(x_test_tfidf)
```

Evaluation

```
accuracy_bow = accuracy_score(y_test, y_pred_bow)
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)

precision_bow = precision_score(
    y_test,
    y_pred_bow,
    average = "weighted"
)
precision_tfidf = precision_score(
    y_test,
    y_pred_tfidf,
```

```

        average = "weighted"
    )
    recall_bow = recall_score(
        y_test,
        y_pred_bow,
        average = "weighted"
    )
    recall_tfidf = recall_score(
        y_test,
        y_pred_tfidf,
        average = "weighted"
    )

    f1_bow = f1_score(
        y_test,
        y_pred_bow,
        average = "weighted"
    )
    f1_tfidf = f1_score(
        y_test,
        y_pred_tfidf,
        average = "weighted"
    )

    print("Using Bag of Words:")
    print(f"Accuracy:{accuracy_bow:.2%}")
    print(f"Precision:{precision_bow:.2%}")
    print(f"Recall:{recall_bow:.2%}")
    print(f"F1-Score:{f1_bow:.2%}")
    print("\nUsing TF-IDF:")
    print(f"Accuracy:{accuracy_tfidf:.2%}")
    print(f"Precision:{precision_tfidf:.2%}")
    print(f"Recall:{recall_tfidf:.2%}")
    print(f"F1-Score:{f1_tfidf:.2%}")

```

Using Bag of Words:
 Accuracy:90.42%
 Precision:90.40%
 Recall:90.42%
 F1-Score:90.40%

Using TF-IDF:
 Accuracy:90.32%
 Precision:90.25%
 Recall:90.32%
 F1-Score:90.26%

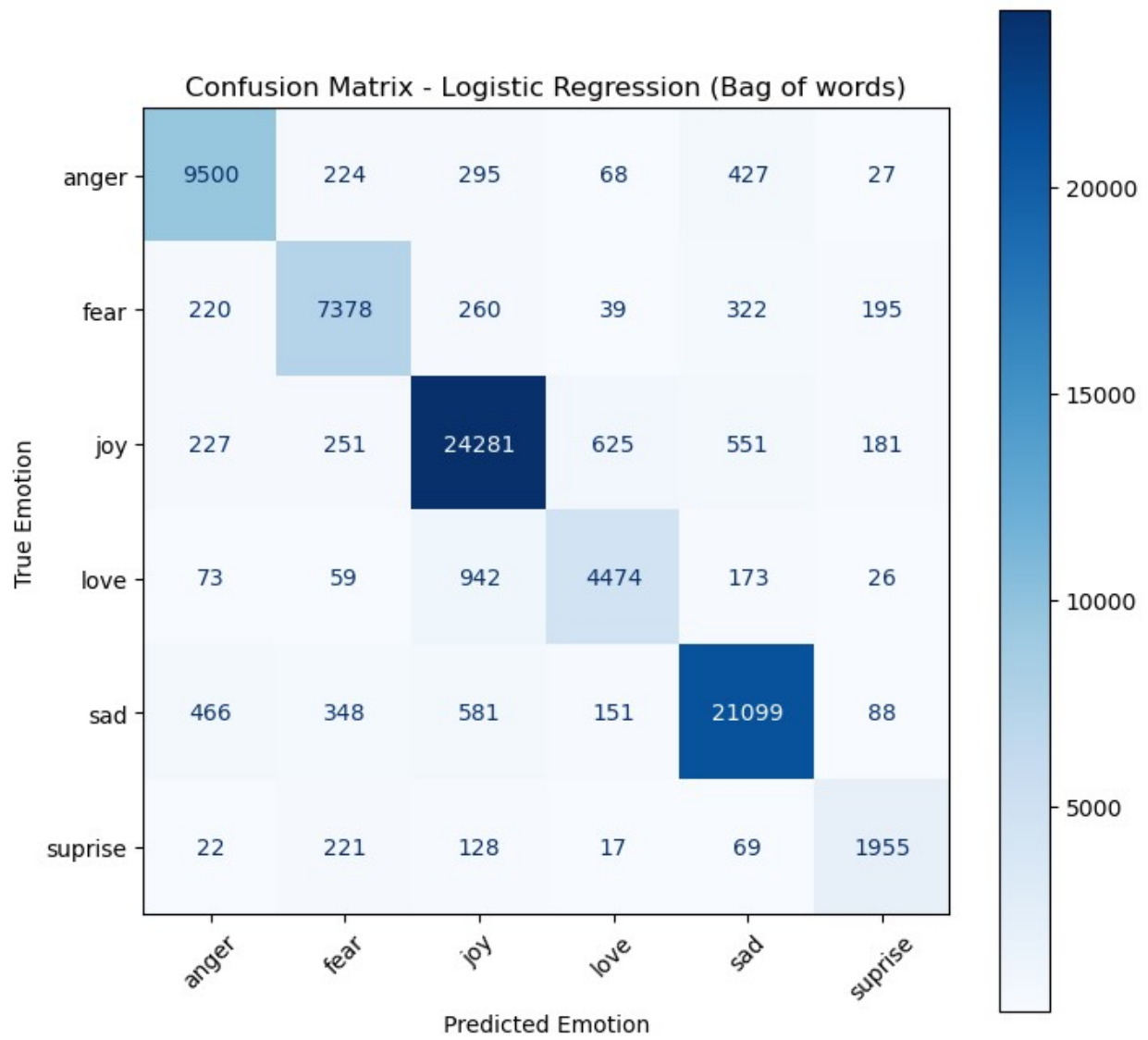
Confusion Matrix

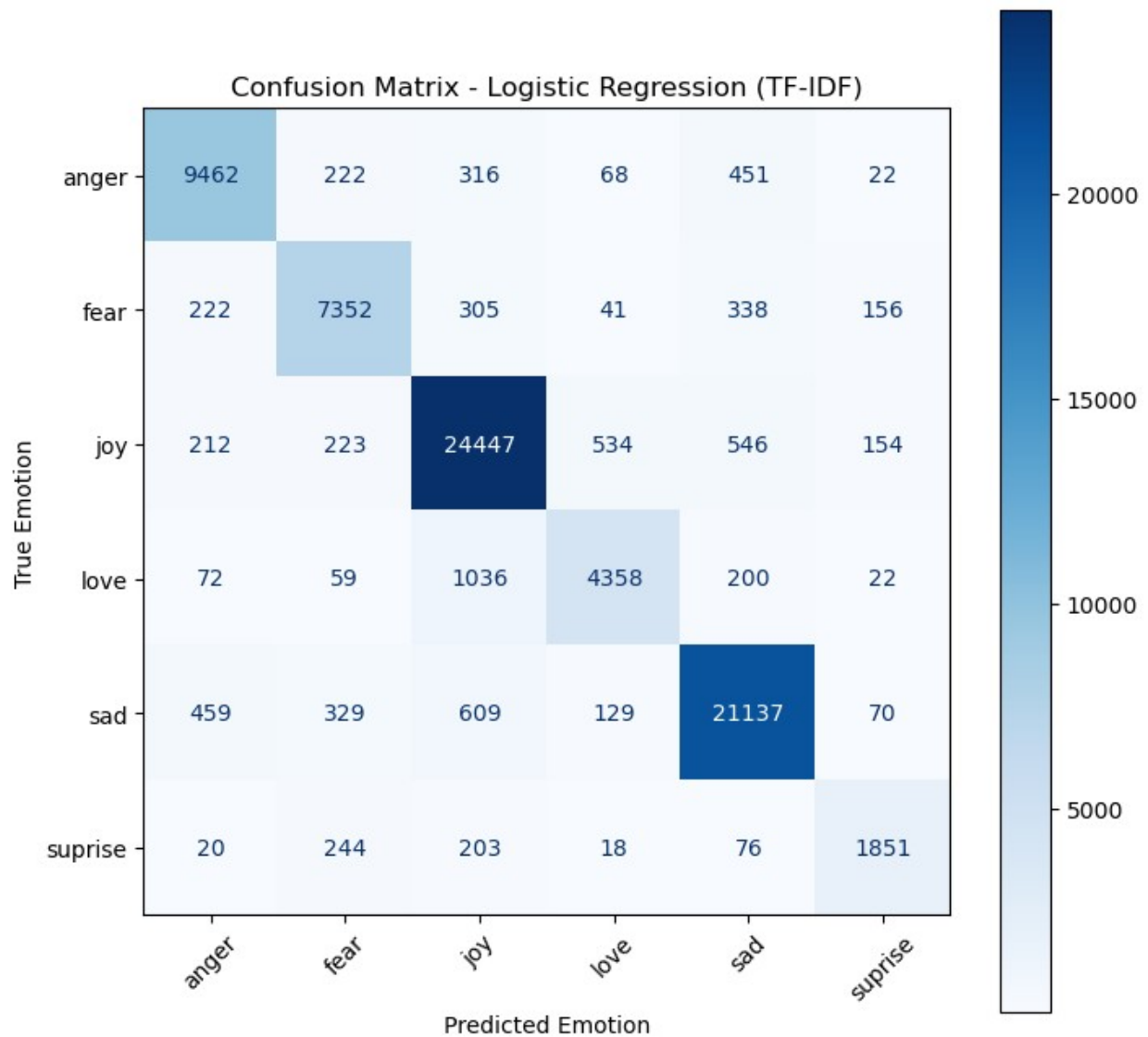
```
emo_labels = sorted(copy_d2["emotion"].unique())
cm_bow = confusion_matrix(y_test, y_pred_bow, labels=emo_labels)
disp_bow = ConfusionMatrixDisplay(confusion_matrix=cm_bow,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize = (8, 8))
disp_bow.plot(ax=ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Logistic Regression (Bag of words)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()

cm_tfidf = confusion_matrix(y_test, y_pred_tfidf, labels=emo_labels)
disp_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_tfidf,
display_labels=emo_labels)
fig, ax = plt.subplots(figsize=(8, 8))
disp_tfidf.plot(ax = ax, cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix - Logistic Regression (TF-IDF)")
plt.xlabel("Predicted Emotion")
plt.ylabel("True Emotion")
plt.show()
```





Nabil

Support Vector Machine (Linear SVC)

Dataset 2

```
X_2 = copy_d2['sentence']
y_2 = copy_d2['emotion']

X2_train, X2_test, y2_train, y2_test = train_test_split(X_2, y_2,
test_size=0.2, random_state=42, stratify=y_2)
```

Bag of Words

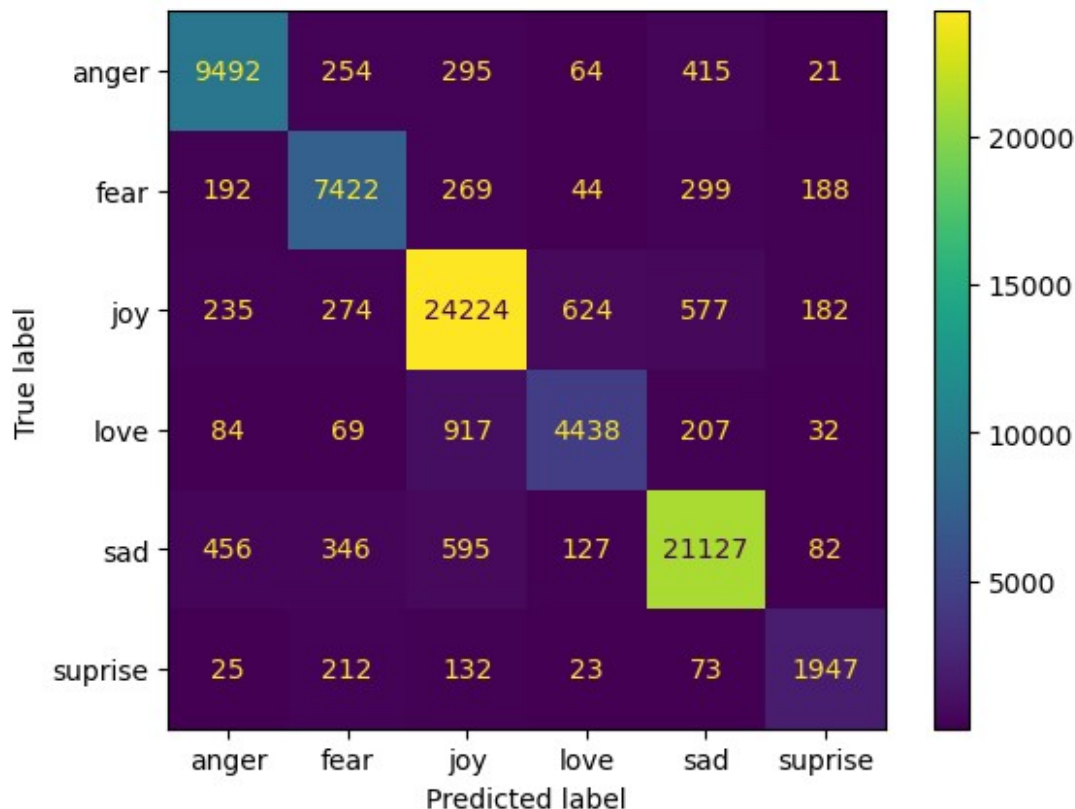
```
bow_2 = CountVectorizer()
X2_train_bow = bow_2.fit_transform(X2_train)
X2_test_bow = bow_2.transform(X2_test)

svm_2_bow = LinearSVC()
svm_2_bow.fit(X2_train_bow, y2_train)
pred_2_bow = svm_2_bow.predict(X2_test_bow)

print("Accuracy: ", accuracy_score(y2_test, pred_2_bow))
print("Precision: ", precision_score(y2_test, pred_2_bow,
average='weighted'))
print("Recall: ", recall_score(y2_test, pred_2_bow,
average='weighted'))
print("F1 Score: ", f1_score(y2_test, pred_2_bow, average='weighted'))

Accuracy:  0.9037294472308888
Precision:  0.9034395340252249
Recall:  0.9037294472308888
F1 Score:  0.9034708529643645

cm_2_bow = confusion_matrix(y2_test, pred_2_bow)
disp_2_bow = ConfusionMatrixDisplay(confusion_matrix=cm_2_bow,
display_labels=svm_2_bow.classes_)
disp_2_bow.plot()
plt.show()
```



TF-IDF

```
tfidf_2 = TfidfVectorizer()
X2_train_tfidf = tfidf_2.fit_transform(X2_train)
X2_test_tfidf = tfidf_2.transform(X2_test)

svm_2_tfidf = LinearSVC()
svm_2_tfidf.fit(X2_train_tfidf, y2_train)
pred_2_tfidf = svm_2_tfidf.predict(X2_test_tfidf)

print("Accuracy: ", accuracy_score(y2_test, pred_2_tfidf))
print("Precision: ", precision_score(y2_test, pred_2_tfidf,
average='weighted'))
print("Recall: ", recall_score(y2_test, pred_2_tfidf,
average='weighted'))
print("F1 Score: ", f1_score(y2_test, pred_2_tfidf,
average='weighted'))

Accuracy:  0.906941537327383
Precision:  0.9064632931987588
Recall:  0.906941537327383
F1 Score:  0.906499234000669

cm_2_tfidf = confusion_matrix(y2_test, pred_2_tfidf)
disp_2_tfidf = ConfusionMatrixDisplay(confusion_matrix=cm_2_tfidf,
```



```
display_labels=svm_2_tfidf.classes_  
disp_2_tfidf.plot()  
plt.show()
```

